

Motivations, Challenges, Best Practices, and Benefits for Bots and Conversational Agents in Software Engineering: A Multivocal Literature Review

STEFANO LAMBIASE, SeSa Lab – University of Salerno, Italy

GEMMA CATOLINO, SeSa Lab – University of Salerno, Italy

FABIO PALOMBA, SeSa Lab – University of Salerno, Italy

FILOMENA FERRUCCI, SeSa Lab – University of Salerno, Italy

Bots are software systems designed to support users by automating specific processes, tasks, or activities. When these systems implement a conversational component to interact with users, they are also known as **conversational agents** or *chatbots*. Bots—particularly in their conversation-oriented version and AI-powered—have seen increased adoption over time for software development and engineering purposes. Despite their exciting potential, which has been further enhanced by the advent of Generative AI and Large Language Models, bots still face challenges in terms of development and integration into the development cycle, as practitioners report that bots can add difficulties rather than provide improvements. In this work, we aim to provide a taxonomy for characterizing bots, as well as a series of challenges for their adoption in software engineering, accompanied by potential mitigation strategies. To achieve our objectives, we conducted a *multivocal literature review*, examining both research and practitioner literature. Through such an approach, we hope to contribute to both researchers and practitioners by providing (i) a series of future research directions to pursue, (ii) a list of strategies to adopt for improving the use of bots for software engineering purposes, and (iii) fostering technology and knowledge transfer from the research field to practice—one of the primary goals of multivocal literature reviews.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software system structures**; • **Human-centered computing** → *Interaction design*.

Additional Key Words and Phrases: bot, chatbot, software engineering, literature review

1 INTRODUCTION

In the last decade, continuous human activities have increased, both in private and working life, raising the need for automation. Consequently, the adoption of **bots** [135, 151]—i.e., software systems designed to automate a specific function or set of activities—has grown and is now available for a plethora of purposes [22, 142]. In particular, **conversational agents (CAs)** [142, 150, 151]—also known as *chatbots*—are bots that communicate with users via natural language or similar methods, using a communication channel. Recently, CAs—and, more generally, bots—have started to be adopted in the software development field [135, 147]. For example, practitioners use bots to automate software maintenance and evolution activities. Similarly, they adopt bots for collaboration, speeding up communication and knowledge-sharing. Beyond proposing them, the research community has also worked on frameworks to ease the development process of conversational systems.

The exploration of bots and CAs in systematic literature reviews (SLRs) reveals a focus on both technological and strategic applications within software engineering (SE).¹ Lewandowski et al. [136] and Suhaili et al. [152] emphasize the technical construction and advocate for considering interaction design and natural language capabilities. Motger et al. [142] and Del Carpio and Angarita [116] discuss integration into user interactions and software development.

¹From now and for the rest of the paper, we will use the term “bot” to refer to both bots without and with a conversational component (CA) when it is not necessary to put in evidence the conversational component.

Moguel-Sánchez et al. [140] examine practical applications, such as project management and static code analysis. A common finding is that, despite their potential, bots, especially with Generative AI and Large Language Models, present development and integration challenges.

To support both the research community—by providing new research opportunities—and practitioners—by distilling knowledge for adopting bots and CAs—we conducted a *multivocal literature review (MLR)* [121, 143], following the guidelines of Garousi et al. [121]. Our aim was to understand the current role and impact of bots and CAs in the SE field, with a focus on (1) motivations for their adoption, (2) associated challenges, (3) best practices, and (4) benefits derived from their adoption.²

Our work both complements and diverges from existing literature by reviewing *formal literature (FL)* and *grey literature (GL)*, while also identifying best practices for bot design and adoption. By addressing engineering and interaction design challenges, we provide guidelines to enhance the practical application of bots in the industry. This research aims to be valuable for both researchers and practitioners. Moreover, the decision to conduct an MLR rather than an SLR for studying bots in SE is supported by Garousi et al. [121], given the rapid technological evolution driven by practitioners and the limited number of academic publications on the topic. Incorporating GL allows for current insights [145, 152] into the practical application of bots, helping to understand their real-world impacts. Aligning academic research with practical experiences provides a comprehensive synthesis of knowledge [121], while including practitioner-driven insights is crucial to avoid publication bias [145, 152].

We collected 107 studies—79 formal studies and 28 grey ones. Results show that CAs are complex to design and deploy, despite the availability of tools to support their development. Moreover, we found that these systems often encounter various issues that hinder their adoption, with interaction problems—such as interruptions and noise—emerging as particularly critical. Nevertheless, our findings indicate that bots and CAs benefit practitioners significantly, justifying efforts to increase their adoption. In terms of contributions, we provide the following:

- A **taxonomy** to categorize and describe bots and CAs based on the motivation of their use for software development and engineering purposes.
- A **list of challenges** associated with adopting bots and CAs.
- A **set of best practices** for managing bots and CAs, along with an association between these practices and the challenges identified (as reported in the Discussion section).
- A **list of benefits** associated with adopting bots and CAs in the context of software development.
- An **online appendix** [134] containing all our findings and all the data gathered during the steps of our MLR.

These findings can be helpful for researchers by proposing new research opportunities and standardizing the terminology for referring to bots and CAs, and for practitioners by providing best practices and knowledge to facilitate the adoption and development of conversational systems.

2 BACKGROUND AND RELATED WORK

According to Storey and Zagalsky [22, 85, 151], a *software bot* is “a conduit or an interface between users and services, typically through a conversational user interface” [151], used for automating processes/activities to improve productivity [22, 151]. For this reason, software bots are widely employed for various purposes. For example, in software development, they have been adopted for automating tasks, such as application deployment—e.g., building, testing,

²Common acronyms in the paper are: SE (Software Engineering), CA (Conversational agent), SLR (Systematic literature review), MLR (Multivocal literature review), FL (Formal literature), and GL (Grey literature).

and reporting—or for supporting developers in communication activities, such as managing agendas and retrieving information [22, 32, 40].

CAs—also known as chatbots—are a specific type of software bots that use communication platforms like *Slack*, *Teams*, or *Discord* to interact with users, generating human-like conversations [3, 157].

The properties of bots and CAs, as well as their potential applications in various fields, have attracted the interest of multiple research communities—e.g., SE and Human-Computer Interaction—in recent years. Although the research on this topic is relatively new, we have identified six systematic literature reviews that synthesize state of the art on the use of bots and CAs in the SE domain [136, 140, 142, 145] and in related fields [152].

Lewandowski et al. [136] conducted an SLR to provide a structured overview of how CAs might be used strategically, including within work and organizational processes, as well as governance structures. The authors identified 21 relevant studies from 2015 to 2020. Their key findings suggested that most research focused on the technical aspects of CAs, while other perspectives should be considered for designing more practical and usable CAs. These aspects include (1) natural language capabilities and training and (2) interaction design—specifically, how a conversational agent interacts with its stakeholders. According to Lewandowski et al. [136], the lack of analysis on these aspects may hinder the broader adoption of CAs in the industry. Our work shares the goal of Lewandowski et al. in eliciting best practices for designing CAs. However, we also extend the analysis to include themes related to motivations, challenges, and benefits of adopting CAs and bots in SE. Additionally, our scope is broader since we incorporate resources from the GL, thus including complementary perspectives. As a result, we provide a more comprehensive analysis of this topic.

Suhaili et al. [152] investigated the technology employed to develop CAs. Indeed, they primarily focus on the components and techniques used to design those tools, other than on the datasets for their training, evaluation metrics, and application domains. Their review covered the scientific literature from 2011 and 2020. The authors reported that deep and reinforcement learning represent popular methods for understanding users' intents and generating appropriate responses. In addition, they found that TWITTER³ and AIRLINE TRAVEL INFORMATION SYSTEMS⁴ represent the most widely used datasets to train and test the capabilities of CAs. Finally, metrics such as accuracy, F1-score, and BLUE index are more frequently used to evaluate the built agents. The work by Suhaili et al. [152] is complementary to ours; besides their goal, we aim to elicit the best practices for designing bots and CAs.

Motger et al. [142] conducted an SLR of secondary literature, reviewing 28 articles. Their goal was to analyze the anatomy of CAs by providing a taxonomy regarding (1) the human-computer interaction features more relevant for users when interacting with CAs, (2) which techniques are used for the design and implementation of CAs, and (3) which approaches are used for the training phase of the NLP and NLU modules of bots. Our work can be considered complementary to the one by Motger et al. [142]. First, we focus on the design of both bots and conversational systems, hence providing a more extensive overview than Motger et al., who only analyzed CAs. More importantly, we approach the literature review from the SE perspective rather than artificial intelligence. As such, we are interested in providing guidelines and recommendations on how to engineer bots and CAs rather than understanding which are the best practices to train their internal artificial intelligence components. Moreover, we elaborate on additional concerns that may inspire further research in SE, such as the definition of a taxonomy to characterize bots and a list of benefits from their adoption. Last but not least, our work is based on a multivocal review [143] and, therefore, we analyze both FL and GL by following the guidelines by Kitchenham et al. [132] and Garousi et al. [121].

³The TWITTER dataset: <https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis>.

⁴The AIRLINE TRAVEL INFORMATION SYSTEMS dataset: <https://www.kaggle.com/datasets/hassanamin/atis-airlinetravelinformationsystem>.

Moguel-Sánchez et al. [140] conducted an SLR to identify (1) the activities for which bots are used in software development, (2) the benefits, and (3) the problems. They reviewed 83 primary studies, and conducted a thematic analysis to organize their quantitative and qualitative findings. They found that bots are mainly used for project management, to automate tasks with a low level of abstraction, assigning team members to code reviews, performing static code analyses, and tracking changes in project repositories. Divergently from Moguel-Sánchez et al. [140], we included in our research also the GL, resulting in a more comprehensive knowledge of the topic other than a larger quantity of literature. Moreover, we also investigated the best practices associated with using bots in SE and created a mapping between challenges and best practices as an ulterior contribution to our work. Finally, regarding the challenges, we rely on already published taxonomy to enhance it and continuously extend the state of the art.

Del Carpio and Angarita [116] conducted an SLR studying the support given by software assistants (SAs)—including bots—to the entire software development process. Specifically, they studied (1) what processes are supported by SAs, (2) how practitioners interact with SAs, (3) what techniques are used to implement SAs, and (4) what challenges face SAs. They reviewed 40 studies and found that most SAs are—actually—CAs and their interaction with users is far from optimal. The work by Del Carpio and Angarita [116] is complementary to ours; besides their goal, we aim to elicit the best practices for designing bots and CAs, and we focus on the benefits of using bots. Moreover, we included the GL, which helped us identify a set of practices to complement the findings of the formal literature.

The study by Biazotto et al. [111] focused on the concept of Technical Debt Management (TDM), conducting a review of 178 studies that identified bots as automation tools for TDM. They proposed a conceptual model to assist practitioners and researchers in addressing the challenges associated with TDM automation. Although their research indicates that increased automation is likely in the near future, further studies are necessary to ensure the effective integration of these tools. Our work adopts a broader perspective on SE and development, aiming to provide a more comprehensive overview of the use of bots across various tasks rather than concentrating solely on TDM.

3 RESEARCH STUDY DESIGN

The *goal* of the study is to understand the current role and impact of bots and CAs in SE, focusing on the (1) motivations for their adoption, (2) challenges, (3) best practices, and (4) benefits provided to software engineers. The purpose is to provide a comprehensive overview of the matter, which might be useful to learn about the current state of the art and of the practice. Moreover, we want to stimulate further research to be pursued. The *perspective* is of both researchers and practitioners: the former are interested in having a unique resource to use as a starting point to deepen their knowledge of the research field; the latter are interested in understanding best practices, challenges, and benefits of using bots.

To achieve our goal, we performed an MLR [143, 146], namely, a research process where past published FL (e.g., journal and conference papers) and GL (e.g., blog posts, videos, and white papers) on the topic investigated are systematically identified, selected, and critically assessed. An MLR builds on top of the concept of SLR [131, 132]—where only the FL is taken into account—to provide benefits not only to the research community but also to practitioners.

We relied on two guidelines when defining the protocol and conducting the review. In particular, concerning the FL analysis, we adopted the well-established approaches by Kitchenham et al. [131, 132]. In addition, we followed the guidelines defined by Garousi et al. [121] when it comes to the GL. Finally, when organizing and reporting the results, we followed the “General Standard” and “Systematic Reviews” guidelines provided by the *ACM/SIGSOFT Empirical Standards*.⁵ In the following subsections, we describe the main steps of each phase.⁶

⁵ACM/SIGSOFT Empirical Standards: <https://github.com/acmsigsoft/EmpiricalStandards>.

⁶The entire process is depicted in Appendix A—Fig. 1 [134].

3.1 Planning the Review

3.1.1 *Initiation Phase.* The Initiation phase consisted of three sub-phases.

Initial Examination of Previous Studies. To collect insights and basic knowledge about the adoption of bots and CAs in SE, we conducted a preliminary investigation in which we reviewed a few existing studies on the matter, e.g., [136, 152]. In particular, we used the *Repository for Bot-related Research* [108]—i.e., a publicly available repository of FL maintained by the SE research community. After reading the studies, we discarded them in a “*throw-away prototyping*”—as suggested in the literature [121, 132]—to start the main review process with basic knowledge, avoiding possible bias or influence from no systematic intervention.

Motivation and Needs Identification. Before choosing to conduct an MLR, Garousi et al. [121] suggested investigating the current state of the art on the topic of interest to identify limitations that GL could address. For this reason, Garousi et al. [121] defined a checklist that consists of seven “Yes” / “No” questions related to the complexity and interest of the topic. A high number of “Yes” justifies the inclusion of GL in the systematic analysis. This step is crucial to prevent the inclusion of GL from being of no benefit to research and resulting in a futile effort. As for our study, Table 1 reports the filled checklist. To fill such checklist, we used the knowledge obtained during the initial phase (described in the previous paragraph) and the SLRs on the topic [136, 144, 145, 152], as suggested by Garousi et al. [121]. Six out of seven responses were “Yes,” which allowed us to decide to proceed with including GL in our study.

- The first two questions were about the topic itself rather than the work. We answered the first question positively since the use of bots in SE is intrinsically a complex topic from both a technical and social perspective [A1–A3]. Moreover, bot technology advances primarily on the practitioner side rather than the research side, thus leading to a fast evolution of technology that is difficult to capture by research studies alone, often requiring slow publication times to facilitate systematic review and process. Furthermore, the broad adoption of bots, coupled with the recent advent of generative AI, leads to a wide range of issues to analyze, many of them from both a research and practitioner perspective. Regarding the second question, since the topic of bots in SE is young, there is not enough literature to determine the presence of a lack of consensus. Nevertheless, the novelty of the topic further poses the need to conduct studies involving the GL as well.
- The third question is about the subject under study in the work. Literature has repeatedly shown that the study context is central to SE research [119, 148]. This becomes even more true when it comes to bots and their impact on practitioners. The present work aims to elicit the motivations, challenges, strategies, and benefits of practitioners’ use of bots for different purposes and in different conditions. Hence, GL’s contribution to characterizing this context is essential. For such a reason, we answered the third question positively.
- Questions four, five, and six are about the work objectives. As mentioned above, the goal of the work is to provide knowledge suitable for (1) opening new avenues of research and (2) improving the use and adoption of technology by practitioners. Given this goal, corroborating academic research with practitioners’ knowledge is essential. In fact, where formal research can provide a systematic and robust set of information, GL can augment that information with concrete and applicable strategies useful to both worlds [121]. On the other hand, academic research may shine precisely in disproving false beliefs born in the practitioner world. In summary, the goal of this paper is to provide knowledge that is useful to both worlds and that is as true and reliable as possible.

Table 1. Questions to decide whether to conduct an MLR [121].

#	Question	Answer
1	Is the subject “complex” and not solvable by considering only the formal literature?	Yes
2	Is there a lack of volume or quality of evidence or a lack of consensus of outcome measurement in the formal literature?	No
3	Is the contextual information important to the subject under study?	Yes
4	Is it the goal to validate or corroborate scientific outcomes with practical experiences?	Yes
5	Is it the goal to challenge assumptions or falsify results from practice using academic research or vice versa?	Yes
6	Would a synthesis of insights and evidence from the industrial and academic community be useful to one or even both communities?	Yes
7	Is there a large volume of practitioner sources indicating high practitioner interest in a topic?	Yes

- Regarding question seven, we found significant evidence about the extensive adoption of bots by practitioners, consequentially increasing the importance of the context on the matter [145, 152]. Beyond that, the professional world’s commitment to process automation has led to the development of many frameworks to support the use, development, and adoption of bots.
- Last, including GL can provide current perspectives and address the gaps in FL. First, it helps to avoid publication bias, although it is noted that the GL accessed may not represent all unpublished studies. Moreover, excluding GL would risk losing critical insights and perspectives on the topic, a conclusion also observed in the case study presented in the guidelines papers [121, 138].

Goals and Research Questions Definition. Our main objective was to provide an overview of the use of bots and CAs for SE purposes. We investigated four aspects that led to multiple research questions to reach our goal.

First, we aimed to investigate the reason behind using bots in SE; our goal was to create a taxonomy to categorize bots based on their use. Hence, we formulated our first research question.

RQ₁—Motivations and Goals

Which motivations and goals cause the use of bots and conversational agents in software engineering?

While the prevalence of bots and CAs has been growing significantly in the recent past [145, 152], developing, adopting, and interacting with them is still problematic [A1, A157]. For this reason, the second research question aimed to categorize and map the challenges faced by practitioners, along with possible solutions.

RQ₂—Challenges

What are the challenges related to using bots and conversational agents in software engineering?

Based on the considerations above, we proceeded with our analysis by extracting a list of known best practices from the FL and GL, with the goal of helping practitioners adopt and develop bots for engineering purposes. Moreover, we aimed to identify possible solutions to the limitations elicited while answering the second research question.

RQ₃—Best Practices

What are the best practices for using bots and conversational agents in software engineering?

Finally, the last research question aimed to investigate the impact of adopting bots, collecting and reporting the benefits of their usage. This led us to the definition of our last research question.

RQ₄—Benefits

What are the benefits of using bots and conversational agents for software engineering?

3.1.2 *Search Phase.* The Search Phase consists of two sub-phases.

Data Sources Selection. This phase aims to identify the most reliable databases to extract the literature and start our process. This step differs between FL and GL.

- *Formal Literature (FL).* To collect the FL, we selected *Scopus*,⁷ *IEEE Xplore*,⁸ and *ACM Digital Library*⁹ as data sources. These databases are considered the top three among the computer science FL sources,¹⁰ other than have been used in various other literature reviews [120, 122, 126, 132]. Furthermore, they were recommended in the guidelines [121, 131] since they can provide a complete overview of the published research. As recommended by [121, 131], we took all the results extracted from the first research and filtered them using eligibility criteria.
- *GL.* To search the GL, we used the Google search engine. The reason behind our choice is that Google (1) has been used by other MLRs on SE [120, 122, 126], and (2) is recommended in various MLR guidelines [109, 121]. However, it is worth noting that deciding when to stop the search process is a complex problem—mainly caused by the high volume of available items [121, 143]. For this reason, we adopted two different choices—as recommended by Garousi et al. [120–122]:
 - We stopped the research when no new concepts emerged from the search results anymore, i.e., *Theoretical saturation*;
 - We applied the *Effort Bounded* that regards the inclusion of the top N search engine hits only. We analyzed the first ten pages produced by Google, observing that its algorithm retrieves and shows the most relevant results in the first few pages.

Search String Design. In this phase, we defined different search strings to collect the FL and GL. In particular, we used the knowledge obtained during the initial examination to draft the search string. Its definition was based on (i) key terms on the topic of interest—gathered from relevant studies, (ii) synonyms and alternative terms, and (iii) logical connectors to combine different terms in different ways.

The initial version of the search string included basic terms such as “bots” and “SE.” During the preliminary exploration phase (Section 3.1.1), we began to use the string and improve it in accordance with the results obtained. We focused on the individuation of synonyms, which led first to the expansion of the search string for FL and then to the detailed definition of the different strings for GL.

As for the FL, the final search string is shown in the box below; we did not include key terms of our research questions, e.g., “goals”, since we aimed to collect as much FL as possible, even if the phase led to high effort.

⁷ *Scopus* website: <https://www.scopus.com/home.uri>

⁸ *IEEE Xplore* website: <https://ieeexplore.ieee.org/Xplore/home.jsp>

⁹ *ACM Digital Library* website: <https://dl.acm.org/>

¹⁰ The top list of computer science research databases: <https://paperpile.com/g/research-databases-computer-science/>

Formal Literature Search String

("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering")

As for the GL, following the guidelines provided by [122, 133], we started from a general search string to define a more specific one based on the goal of each research question. The reason behind our choice arose from the noise present in the output that characterizes informal databases, like the Google search engine. Table 2 reports the search strings for the GL based on the research questions of our study.

3.1.3 Eligibility Criteria Phase. In this phase, the aim is to define a series of criteria to apply after collecting the literature for filtering and identifying those that provide direct evidence based on our research questions [121, 131]. The phase consisted of two sub-phases: (1) Defining the FL eligibility criteria and (2) Defining the GL eligibility criteria.

As for the FL, the eligibility criteria are divided into three sets that had to be evaluated for each paper extracted from the search phase.

- (1) *Exclusion Criteria*, namely, criteria that, if met for at least one of them, decree the elimination of the study from the analysis set;
- (2) *Inclusion Criteria*, namely, criteria that, if met for at least one of them, decree the inclusion of the study into the analysis set;
- (3) *Quality Criteria*, namely, criteria that are used to grade studies and may lead to their elimination if the grade is below a certain threshold.

Table 3 shows exclusion and inclusion criteria. We defined such criteria according to the guidelines by Kitchenham et al. [121, 131] and past literature in the SE field [126, 133]. Each criterion could be only *True* or *False*. Using these filters, we could exclude all preliminary research results, e.g., workshops or posters, and avoid considering duplicated studies derived from the combination of three data sources. Nevertheless, we decided not to exclude workshop studies reported in the Repository for Bot-related Research [108] since these could be highly related to this work's main reference community. In addition, we included EC6 in order to ensure that the collected studies were concerned with the topic of interest. Moreover, as for other LRs [122, 126, 136, 145, 152], we defined our inclusion criteria to match our objective and research questions. Regarding our exclusion criteria, it is worth discussing the EC5—Studies written before 2014. We decided to apply such criteria based on the findings of previous LRs [145, 152], which report that literature on

Table 2. Search strings for GL.

#	Search String	RQ
0	("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering")	General String
1	("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering") AND ("goal(s)" OR "motivation(s)" OR "objective(s)")	RQ ₁
2	("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering") AND ("challenge(s)" OR "limitation(s)" OR "negative impact(s)")	RQ ₂
3	("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering") AND ("best practice(s)")	RQ ₃
4	("bot(s)" OR "chatbot(s)" OR "chat bot(s)" OR "conversational agent(s)") AND ("software engineering") AND ("benefit(s)" OR "positive impact(s)")	RQ ₄

Table 3. Exclusion and Inclusion Criteria for FL.

Exclusion Criteria	Inclusion Criteria
EC1 Studies not written in English.	IC1 Studies that report about the uses of bots in SE.
EC2 Posters.	IC2 Studies that report about the challenges of bots in SE.
EC3 Workshop studies.	IC3 Studies that report about the best practices in the use of bots in SE.
EC4 Study not entirely available for free.	IC4 Studies that report about the benefits of using bots for SE.
EC5 Studies written before 2014.	
EC6 Studies not related to the topic of interest.	
EC7 Duplicated studies.	

software bots—particularly in their communication-oriented form—started growing after 2014. The other exclusion criteria are widespread among other LRs, and some have been applied using the filters integrated into the selected databases for data source selection.

Regarding the quality criteria, we defined them as the following:

- Q1** Is the valuable information for the review clearly reported and indicated in the paper?
- Q2** Are the findings clearly reported in the paper?
- Q3** Are the RQs or objectives clearly defined?
- Q4** How many pages does the paper consist of? (with respect to the top venue formats)

The first three questions could be answered as “Yes”, “Partially”, “No”. Each answer corresponded to a numerical value, i.e., ‘1’, ‘0.5’, ‘0’, respectively. The sum of these values reflects the quality score of the study. If the score was lower than 2, we excluded it from the review. The last one could be answered as “1 to 2”, “3 to 4”, and “4 and up”.

Concerning the *GL* eligibility criteria evaluation, we followed the guidelines proposed by Garousi et al. [121] combining inclusion and exclusion criteria with quality assessment criteria (Appendix A—Table 1).

3.2 Conducting Phase

3.2.1 Studies Selection Phase. In the studies selection phase, the aim was to apply the planned strategy to collect the literature [121, 131]. The analysis included all studies published from 2014 to 4/4/2024 (Appendix A—Fig. 2 [134]).

As for the extraction and selection of the FL, we started by running the search string on the three selected databases, e.g., Scopus, obtaining a set of 2108 studies. We imported the results of the three datasets in Excel, and the first two authors of the paper started the inclusion and exclusion process. As a first filtering step, we started deleting duplicates by the three datasets merged. Then, the remaining set of studies was divided in two, and the first two authors started applying inclusion and exclusion criteria on the two separate sets; the authors determined if they include or exclude the paper starting by reading the title, abstract, and—if necessary—the entire body of the studies. After analyzing 20 studies each, the two authors reviewed corresponding work and discussed; no conflict arose, and then the authors continued applying the exclusion and inclusion criteria, resulting in 173 studies included and 1935 excluded. Next, an ulterior session of filtering was applied jointly by all the authors on the 173 studies that remained; this aimed at identifying potential errors in the inclusion process. After this step, we excluded other 64 studies, obtaining a final set of 109 studies. Finally, the first two authors of the paper applied the quality criteria, obtaining a final set of 79 studies from the FL.

Regarding the *GL*, the first author started using the search strings on the Google Search Engine to collect the literature. For each search string, the first 10 pages were checked, and the items judged related to the work objectives

were imported into an Excel file until theoretical saturation was reached [121]. By doing so, we obtained a set of 52 items. Then, the first two authors of the paper started applying all the eligibility criteria simultaneously, as Garousi et al. [121] suggested. Such a process led to the identification of 28 items for the GL.

After completing the extraction and selection process for both types of literature, we combined the two sets. This final step resulted in a collection of 107 studies, which formed the basis for the subsequent data extraction phase.

Regarding the distribution of literature for type—i.e., blogs (for GL), conferences, workshops, and journals—most of the works are conference studies (52), while 28 are blogs, 13 are workshop studies, and 14 are journal studies. This is not a surprise, seeing that the research topic specifically concerning CAs—is extremely recent and tends to be more relegated to conferences and live discussion settings. Moreover, as expected, a large amount of the literature comes from the *International Workshop on Bots in SE (BotSE)*; it is the main research community on bots and CAs for SE purposes. In addition, there are also top venues on SE, e.g., *International Conference on SE (ICSE)* and *Transaction on SE (TSE)* (Appendix A—Fig.3). More details are in the online appendix [134].

3.2.2 Data Extraction Phase. In the data extraction phase, we started from our objectives and research questions to define a series of data extraction forms, i.e., sets of questions to identify useful elements to answer a research question in the studies [121, 131].

Aligning with other similar works, we defined the extraction forms jointly—through brainstorming and discussion—and then the first author applied them. We identified and extracted the relevant data using a pre-defined data extraction form from each selected source that we needed to answer the research questions. We also extracted some general information—e.g., authors’ names, published year, and keywords—as recommended by guidelines on literature review [121, 131]. To validate and test our forms, we conducted a preliminary extraction on a set of 15 randomly selected sources—like in other literature reviews [120, 122, 126, 139].

From a practical standpoint, we imported all the studies into a new Excel sheet and added a column for each data form identified. Then, we began extracting information from the studies in random order. If the extracted information was different from what had been extracted previously, we would create a new column in the file under the data form group (following a hierarchical structure). However, if the information extracted from a paper had already been indicated by another paper, meaning the column already existed, we would simply mark the corresponding cell with a sign and add any new information. For example, one of the data forms for RQ1 was “what are the reasons for using bots in SE?” Through the paper by Van Tonder and Le Goues [A4], we extracted the information “repair bots, to identify issues in the code and automatically resolve them.” Subsequently, the paper by Monperrus [A5] also reported the same information, so an “X” was added to the corresponding cell in the repair bots column.

All the extracted data are in our online appendix [134]. We maintained traceability between the forms and the research questions as recommended by MLR guidelines [121].

3.2.3 Data Synthesis and Analysis Phase. In the data synthesis phase, we synthesized data previously collected to present them to the target audience in a readable and usable way.

After extracting and dividing the data between the various form groups, the first two authors analyzed them. In this way, we avoided losing important information or misunderstanding data [121, 131]. If the first two authors were unsure how to synthesize a study’s results, the plan was to ask the remaining two authors for help; this case never occurred.

To conduct a correct data synthesis, we relied on two qualitative analysis methods, namely, *narrative synthesis*—i.e., the description and ordering of primary evidence with commentary and interpretation of a set of data—and *thematic analysis*—i.e., the summary of studies under a set of recurrent themes in literature [115]. First, we used narrative

synthesis to perform an initial analysis of the gathered data from the previous phase. Then, we used thematic analysis to classify and categorize the data.

We started with the data extracted in the previous phase and began annotating them with descriptions and labeled comments using keywords such as “Repair bots” and “Maintenance bots”. Subsequently, we identified descriptions that closely resembled each other or appeared to be related and began aggregating them into individual categories. Each category became a new column in the Excel file, and through iterative repetition of this process, some columns were grouped into thematic similarity groups. For instance, “Repair bots” and “Bots used for identifying flaky tests” were grouped under the category test bots. To maintain traceability, within the file, all studies contributing to the generation of a category were marked with an “X” corresponding to the paper and the column dedicated to that category.

The first two authors jointly conducted the steps mentioned above—at least for a subset of the items—to identify possible discussion points to strengthen the final results. At the end of the process, we obtained a new set of insights and information categorized under four macro categories—one for each research question. We reported all our findings in the online appendix for our work [134].

3.3 Results Reporting Phase

Based on the target audience of this review, i.e., practitioners and researchers, we relied on the guidelines provided by Garousi et al. [121] to enhance the readability—particularly from the practitioners’ side. For such a purpose, we (1) decided to include a practical-oriented Discussion section (Section 5.3), (2) asked practitioners for feedback on our results, and (3) adopted various tools to enhance results summaries.

4 ANALYSIS OF THE RESULTS

In the following section, we describe the results achieved for each research question and report some statistics of our papers collection.

4.1 RQ₁: On the Use of Bots and Conversational Agents for Software Engineering Purposes

As a first goal, we aimed to identify the motivations and objectives behind the adoption of bots for SE purposes. Our purpose was to gather general and fundamental pieces of information to (1) define a taxonomy to categorize bots based on their use and (2) better perform the following research step.

Our investigation into answering the first research question (described in Section 3) led to the identification of 13 fundamental motivations for adopting bots and CAs in SE. In order to provide a more comprehensive and understandable taxonomy, we categorize each of the fundamental motivations into 4 macro categories of bots (Depicted in Figure 1 and in Appendix B—Fig. 1 [134]). The identified motivations are not mutually exclusive; the same bot can be used for different motivations according to a plethora of reasons, e.g., working roles, needs, and types of projects. In the following section, we elaborated on each category and associated motivations, corroborating the discussion with the primary studies and GL supporting them.

4.1.1 Software Product Bots. In this category, we group all the bots and associated motivations of use used to perform automatic operations on source code or related artifacts and activities. We identified the following sub-categories:

Development. This sub-category refers to bots that *are used to generate source code instead of developers or other practitioners*. In our investigation, we found 11 studies, 9 from the FL [A6–A13] and 2 from the GL [B80, B81] ($\approx 10.28\%$ of the entire dataset).

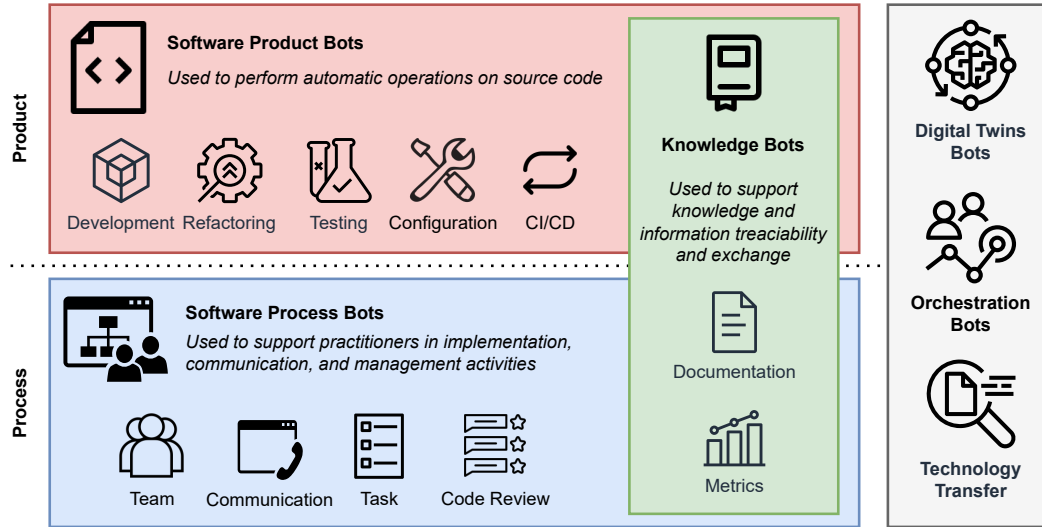


Fig. 1. Motivation for adopting bots and CAs.

Part of the literature focuses on the use of bots (in most cases empowered by Generative AI) to automatically write source code [A9, A13][B80, B81] and code for machine learning purposes [A9]; in this way, practitioners can save time and increase productivity. Nevertheless, Generative AI is still raw in this task, and software engineers do not rely heavily on it [A9, A13][B80, B81]. Another part of the literature combined bots and models-based language to generate source code and entire applications [A6, A7, A10–A12, A14]; in this context, bots act like facilitators in creating models representing complex systems and modules. Interestingly, bots to generate smart contracts [A7, A12] and recommendation systems [A11] (thus, bots used to generate other bots) are being proposed.

Refactoring. This sub-category refers to bots that are used to (1) *improve source code* and (2) *avoid and/or resolve technical debts by means of refactoring operations*, i.e., *Refactoring Bots*. Our dataset has 11 studies, 8 from the FL [A9, A15–A21] and 3 from the GL [B82–B84] ($\approx 10.28\%$ of the entire dataset).

Some of them are related to technical debt [A15, A17]; for example, bots in such a context are used to refactor code smells automatically. Moreover, bots can also be used to support quality aspects of source code, e.g., by means of quality criteria [A16] or code readability [B82], and provide fixes when static analysis tools detect violations [A18, A19]. In the recent past, refactoring has also been automated using Generative AI, even without satisfying results [A9].

Testing. This sub-category refers to bots that *support testers and similar roles during the testing activity and for executing test code*. Our dataset has 11 studies, 10 from the FL [A4, A5, A17, A19, A22–A27] and 1 from the GL ($\approx 10.28\%$ of the entire dataset).

Most of the literature regards the use of bots for automation of test cases execution [A4, A5, A17, A22, A23, A26, A27]. Furthermore, we found that many bots are used to identify bugs and perform automatic fixes [A4, A5, A17, A19, A22, A23][B82], i.e., *Repair Bot*. Last, some works propose bots to (i) support developers in writing test code [A27] and (ii) implement heuristics to detect *flaky tests* [A22]—i.e., tests exhibiting both a passing and failing behavior when run against the same code [137].

Configuration. This sub-category concerns bots used to support practitioners during the setup and configuration processes of new software projects. Our dataset has 6 studies, 4 from the FL [A19, A22, A28–A30] and 1 from the GL [B84] ($\approx 5.61\%$ of the entire dataset).

From the identified literature, two studies propose bots for automatically creating/modifying software configuration files or similar [A28, A29], and four studies [A19, A22, A30][B84] report about bots used to manage and monitor project dependencies.

CI/CD. This sub-category regards the usage of bots *to support everyday activities in the context of CI/CD*. In our investigation, we found 8 studies, 4 from the FL [A22, A25, A31, A32] and 4 from the GL [B82, B84–B86] ($\approx 7.48\%$ of the entire dataset).

Some studies analyze the usage of bots for monitoring software systems (for example, on GitHub) and performing automatic operations if certain conditions are met (e.g., automatically build the system on branch merge) [A22, A25]. Finally, five works study the use of bots to automatically trigger actions when specific criteria—related to no-code platforms—are met [A22, A31, A32][B82, B85]. For example, automatically creating an issue on GitHub when a bug is reported, or a task is assigned in the collaboration platform of the team (e.g., *Trello* and *Jira*). Moreover, some bots to support DevOps-related activities started to emerge [B86].

4.1.2 Software Process Bots. Under this category, we group all the bots used to improve and enhance communication, collaboration, and management activities. We identified the following three sub-categories:

Team. This sub-category refers to bots used *to support practitioners in handling team quality attributes like turnover and socio-technical aspects*. We found 6 studies, 3 from the FL [A33–A35] and 3 from the GL [B82, B85, B87] ($\approx 5.61\%$ of the entire dataset).

Regarding the motivations, some bots can be used to support management activities such as risk management [B87] and *community smells* management [A34], i.e., anti-patterns precursors of social debt [153]. Moreover, a study proposes to use bots specifically to reduce *turnover*—the phenomenon of continuous influx and retreat of human resources in a team [117]—in an open-source software community by improving the reviewer selection process in code review [A33]. More related to the open-source communities, bots are used to help onboard new contributors [B82] and alleviate burnout and stress [A35][B85] by—for example—supporting core contributors’ activities.

Communication and Collaboration. This sub-category concerns bots used *to help communication and collaboration between practitioners*. During our investigation, we found 8 studies: 7 from the FL [A6, A8, A33, A36–A39] and 1 from the GL [B85] ($\approx 7.48\%$ of the entire dataset).

Two studies reported that practitioners use bots to facilitate meetings and communication in community chats, encouraging team participation [A37][B85]. Bots can also assist managers with scheduling and planning meetings [A36]. Additionally, bots help detect when two individuals work on the same artifact, preventing code conflicts [A38]. Bots can record past decisions to avoid information loss and support future decisions [A6, A8]. They also identify experts to reduce turnover and assist newcomers [A33, A39].

Task. Practitioners adopt bots *to support activities related to task management*, e.g., evaluation, assignation, and planning. During our investigation, we found 5 studies: 3 from the FL [A36, A40, A41] and 2 from the GL [B87, B88] ($\approx 4.67\%$ of the entire dataset).

Some of the bots are adopted to find the right person for the right task by using metrics and developers profiling [A40, A41][B87]. More related to the managers, a work proposed an all-round bot for task management assistance; the

focus of such a bot is task monitoring, issues management, planning, and also discussion of task characteristics to assigning it to the right person [A36].

Code Review. The review revealed that practitioners use bots to support the code review process, often associating it with pull requests. Our dataset has 6 studies on this topic, 5 from the FL [A33, A42–A45] and 1 from the GL [B85] ($\approx 5.61\%$ of the entire dataset).

In general, researchers report that the adoption of such bots could effectively benefit software projects in different ways. [A33, A42, A44] For example, turnover decreases, and communication becomes more effective; moreover, pull requests increase in quality, and contributions to open-source projects are more rapid than without the bots. Nevertheless, the risk of introducing noise exists; some studies proposed the adoption of bots to orchestrate code review bots and summarize their activity and outputs more concretely and cleanly [A43, A45].

4.1.3 Knowledge Bots. Under the category Knowledge Bots, we group all the bots used to store, share, and manage information and knowledge about the software project. We identified the following sub-categories:

Documentation. Practitioners use bots *to support documentation writing*. Our dataset has 10 studies regarding this type of bot, 9 from the FL [A6, A7, A22, A24, A28, A29, A46–A48] and 1 from the GL [B85] ($\approx 9.35\%$ of the entire dataset).

Several works focus their attention on bots used to automatically create and update documenting artifacts [A6, A28, A29][B85] (e.g., code comments, documents, and reports). The state of the art [A7, A22] has proposed bots to automatize the writing of notes for new software product releases, using NLP to synthesize developers' commit messages and comments. Moreover, practitioners started to use Generative AI to write code commits messages [A47, A48]. Other studies propose bots to improve and speed up agile processes by automatizing the agile retrospective step by collecting metrics from different sources [A7, A46]. Last, bots are also used to record and document design decisions and, for some of them, generate static or behavioral models on-the-fly [A6, A7] (e.g., UML graphs).

Metrics. Bots in the Metrics category are used *to monitor the team environment (both product and process) and collect metrics*. We found 7 studies related to this category, 4 from the FL [A22, A25, A49, A50] and 3 from the GL [B86–B88] ($\approx 6.54\%$ of the entire dataset).

Some studies are related to collecting and analyzing product and process metrics, allowing practitioners to make informed decisions [A49, A50][B87, B88]; this is particularly important when dealing with services and micro-services applications [A50]. Moreover, also in this case, bots seem to be used as managers' peers to improve the overall management process [B87, B88]. Furthermore, some bots have also been adopted to support DevOps activities by computing metrics and suggesting ways to improve the pipeline [A49][B86].

4.1.4 Emergent Bots. In this last category, we have included three reasons for adopting bots that have been emerging in recent times and that we believe are worthy of mention, although still in an embryonic stage.

Digital Twins Bots. Bots in this category are integrated into the development workflow *to support specific developers and act as companions*. We identified 19 studies about this type of bot, 16 from the FL [A17, A22, A24, A25, A35, A51–A61] and 3 from the GL [B82, B85, B88] ($\approx 17.76\%$ of the entire dataset).

The majority of works report that bots are used to support the retrieval of information regarding general implementation questions and, particularly, API documentation and guidelines [A22, A52, A53, A56–A58][B85]. Then, some studies focus on using bots to enforce the application of formatting guidelines or requirements [A17, A59][B82] and

good programming practices [A51][B82]. Furthermore, some bots are also used like actual peers during the programming activity (pair-programming bots) [A60, A61]; among these, some are used to detect possible implementation choices that could lead to defects in the code and propose solutions [A55]. Related to the open-source field, some bots act like peers to recommend useful tools to solve a problem [A54], support elite developers in general tasks, and help them stay focused on the work by eliminating noise [A35].

Orchestration Bots. Despite being useful for practitioners, a well-known reported problem with bots is the *noise* arising from their use. In short, since bots are still unable to behave according to the operational context, if their interaction is not well designed, it could result in them providing too much information to practitioners, often unhelpful, causing distraction and discomfort [A1, A25, A63, A135][B85]. To solve this problem, some practitioners operationalize bots specifically designed *to orchestrate and summarize the results of other bots*. In such a sense, these Orchestration Bots act like bot managers. We identified 2 studies about this type of bot, both in the FL [A43, A45] ($\approx 1.87\%$ of the entire dataset).

Wessel et al. [A43] proposed a series of guidelines for developing these types of bots, while Ribeiro et al. [A45] created FUNNELBOT, a bot designed using the previously mentioned guidelines to orchestrate pull-request bots.

Technology Transfer Bots. Last but not least, and more attractive for researchers, some bots are starting to be used *to support the technology transfer between research and the practitioner field*. Indeed, the limit of research-developed tools is often the poor effort invested in developing their non-functional requirements, resulting in poor quality. Some bots are starting to be developed specifically to surpass such a limitation. We identified 3 studies about this type of bot, 2 in the FL [A31, A34] and 1 in the GL [B85] ($\approx 2.80\%$ of the entire dataset).

In this sense, Beschastnikh et al. [A31] and Voria et al. [A34] proposed an ecosystem of bots specifically designed to integrate other research-originated bots and make them more easily available to and usable by practitioners.

4.2 RQ₂: On the Challenges of Bots and Conversational Agents for Software Engineering Purposes

The diffusion of bots and CAs for supporting software development processes has also led to the emergence of new challenges for computer scientists. For this reason, we investigated the challenges related to the interaction with, adoption, and development of bots for SE purposes.

Regarding the existing literature, Wessel et al. [157] identified a series of challenges in the interactions with software bots in the open-source development context, with a particular focus on the concept of “*noise*”—namely, annoying bot behaviors, e.g., verbosity and unsolicited actions, which lead to deterioration of communication in teams. In terms of contributions, the authors provided a series of challenges experimented by practitioners, categorized under three main categories, i.e., *Interaction*, *Adoption*, and *Development Challenges*.

On the one side, we found challenges already covered by Wessel et al. [157]. On the other side, we contributed state of the art with new challenges—indicated in Figure 2 in bold and red text color. Specifically, we decided to maintain the high-level separation proposed by Wessel et al. [157]—the left side of Figure 2—and to re-design the low-level categorization—the right side of the figure—based on the findings of our MLR. The following subsections report each identified challenge in detail.

4.2.1 Interaction Challenges. Most of the challenges identified by the literature regard the interaction between practitioners and bots. Such interaction can be intended as unidirectional—the user receives a message from the bot and reacts to it—and bidirectional—the user and the bot actively exchange messages. During our investigation, we found 27 studies;

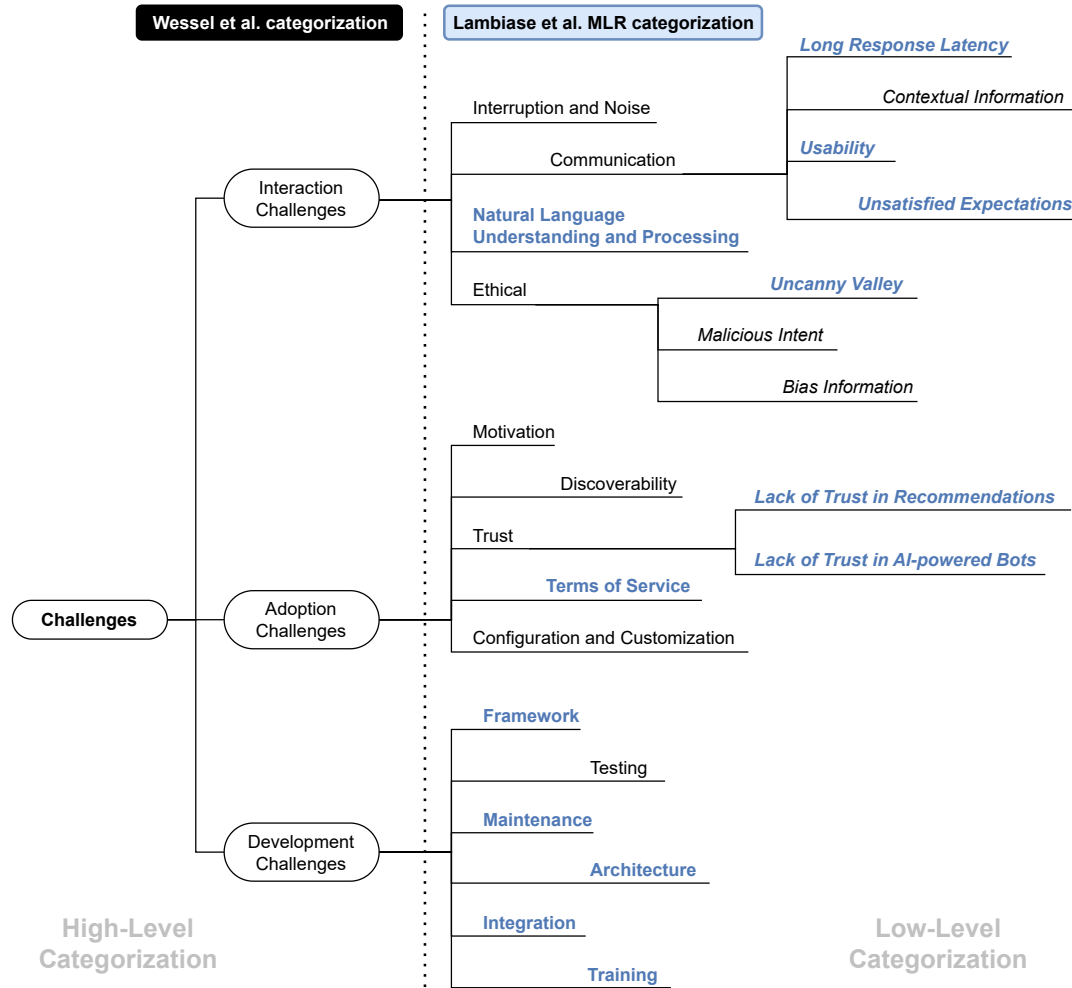


Fig. 2. Categorization of challenges associated with bots in the context of SE. This categorization is based on the structure proposed by Wessel et al. [157]. New challenges originated from our work are shown in blue.

17 from the FL [A1, A3, A19, A25, A39, A43, A61, A63–A71, A135] and 10 from the GL [B81, B85, B86, B89–B95] that report on this challenge ($\approx 25.23\%$ of the entire dataset).

Interruption and Noise. One of the most frequently noted challenges involves certain bots exhibiting a tendency towards verbosity or producing irrelevant output [A1, A25, A43, A63, A70, A135][B85]. This issue, commonly referred to as “noise” in the literature, can significantly affect bot usability as well as team communication and collaboration. The origin of this behavior often lies in the bot’s design—sometimes, bots are intentionally programmed to “harass” developers. On the other side, errors in the bot’s operation can also result in unexpected and disruptive noise.

Communication. Another challenge refers to the communication between the user and the bot [A1, A3, A19, A25, A39, A64–A68, A70, A71][B81, B85, B86, B89, B90, B92, B93]. For example, **ensuring a short response time** comes to be mandatory for bot use: various bots—especially those based on API calls or off-the-shelf modules—are characterized by long responses to users’ inputs and consequent frustration in them [A65][B90]. Nevertheless, architectures of this type are often recommended for these software systems, and developers are faced with a trade-off between high modularity and maintainability—given by using such architectures—and fast response time. To continue, several works report the difficulty of bots in **providing practical and contextualized information**: indeed, bots should have information about the environment in which they are operating (e.g., repositories address, commit template, and team composition), but this knowledge is difficult to collect automatically [A1, A19, A67, A70][B85, B90] and is often limited by technology limitations of Generative AI tools [A71][B93]. Another primary concern regards how bots communicate with the user, specifically in the conversation design, generation of reply messages, and design of the chatbot’s graphical user interface [A1, A3, A25, A39, A64, A66–A68][B85, B89, B92]. Moreover, most practitioners struggle to **provide straightforward and satisfying user interfaces**, and researchers underline the **lack of guidelines and standards for bots usability** [A1, A3, A25, A39, A64, A66–A68][B81, B85, B89, B92]. The last challenge is related to the use of Generative AI-powered chatbots; this technology is relatively recent and, like many other disruptive technologies, is at the point of maximum expectation and illusion about its actual use by practitioners based on the Gartner hype cycle. However, the technology still needs to be updated to meet user demands, specifically regarding SE professionals. This aspect causes **disillusionment and frustration because of unsatisfied expectations** when interacting with such types of chatbots [B86].

Natural Language Understanding and Processing. As already described in Section 2, CAs’ interactions are based and modeled on the concept of *intent*, i.e., a conceptual representation of a possible user request characterized by a set of mandatory information to fulfill the request (called *entities*). Their correct interpretation is crucial for the bot operation and is often based on text interpretation using natural language processing (NLP) and understanding (NLU) algorithms. Therefore, **training NLP and NLU machine learning models** represented the most mentioned challenge [A3, A39, A61, A68, A69][B81, B86, B91, B94]; the main problem lies on the fact that a conversational agent works in singular contexts, and it is difficult to (1) extract sentences to train models and (2) reuse sentences from other contexts. For example, practitioners tend to be **skeptical because of past experiences with bots exposing malicious intents**—e.g., theft of sensitive information or insertion of malicious code. So, interacting with these bots can be challenging because their identification is not always an easy task [A1]. In addition, another challenge is related to a well-known topic such *fairness* [156], intended as the equitable and just treatment—without discrimination—assumed by software systems—particularly in the context of artificial intelligence. Specifically, CAs are based on trained machine learning models using datasets containing sentences and assertions by potential users, thus possibly leading the bot to be biased given the different cultures and backgrounds of the individuals who built the dataset, which could **lead to unexpected and often discriminatory behaviors** [A1][B81, B94, B95]. Last but not least, the literature highlighted the phenomena known as *Uncanny Valley*—i.e., the phenomenon whereby a computer behaving like a human being evokes a sense of unease and revulsion in the person interacting with it [130, 141]. The phenomenon is extremely diffused, especially in the interaction with CAs. It is related to the humanization of robots: people tend to exhibit unpleasant feelings—such as revulsion and uneasiness comparable to perturbation—when interacting with machines with overly “human” attitudes. Managing this phenomenon so that **the bots turn out to be human enough to improve communication without being disturbing** is a difficult challenge [A1, A39][B85].

4.2.2 Adoption Challenges. Several studies reported challenges in adopting bots and chatbots for development reasons. We found 19 studies: 12 from the FL [A1, A4, A5, A20, A22, A25, A30, A54, A60, A72–A74] and 7 from the GL [B81, B84–B86, B92, B94, B95] that report on such a category ($\approx 17.76\%$ of the entire dataset).

Motivation and Trust. According to various studies, the first challenge involves **motivating practitioners to use bots** [A1, A5, A25, A74][B84–B86, B92]. In fact, various practitioners are skeptical of bots because of bias—they think they can implement malicious behaviors or are useless and noisy, underlining a **lack of trust** [A5, A20, A25, A30, A54, A60, A72–A74][B84–B86, B92] of developers in bots’ recommendations. Furthermore, **the problem of trust becomes harder when dealing with Generative AI-powered bots**. In fact, such AI is notoriously subject to hacking actions performed by malicious persons and aimed at influencing their behaviors [B94, B95] by introducing bugs or defects in the source code [B86]. Furthermore, the risk of unexpected privacy violations [B86] and the lack of transparency in personal data management [B81, B94] constitute a severe limit to the adoption of Generative AI-powered bots.

Discoverability and Customization. Furthermore, **identifying the correct bot to solve a problem is challenging** [A1, A22] for two reasons: first, there are limited search mechanisms to identify appropriate bots given a problem; second, the lack of a unified standard to characterize bots leads to difficulties in their description and categorization. This becomes even more problematic when practitioners identify the correct bot but have **difficulty configuring it** [A1, A4, A30][B86]. In fact, such systems often have few configuration options, even if they have to be used in significantly different contexts.

Terms of Services. The adoption of bots can be **hindered by the specific terms of service of the platforms they are intended to operate on**, which may restrict their functionalities [B84]. This legal and regulatory environment can severely limit the actions that bots are allowed to perform, thereby reducing their effectiveness and potential benefits. As a result, practitioners often hesitate to integrate bots into their workflows due to concerns over compliance and the potential legal ramifications of bot activities that violate these terms. This fear of inadvertently breaching platform policies can lead to a reluctance to adopt bot technology despite its potential to enhance productivity and efficiency.

4.2.3 Development Challenges. The last category of identified challenges concerns the development of bots. Specifically, we found 19 studies, 13 from the FL [A1–A3, A26, A39, A48, A61, A64, A68, A72, A75–A77] and 6 from the GL [B90, B96–B100] that report on such a category ($\approx 17.76\%$ of the entire dataset).

Framework, Architecture, and Integration. First of all, various actors have proposed various frameworks to support developers in the development of bots and chatbots (e.g., Microsoft Azure Bot Framework, Google DialogFlow, Amazon Lex, and Rasa). Although such variety can provide practitioners with a large plethora of possibilities to reach their objectives, the literature identifies a **difficulty in identifying the correct framework to start development** [A1, A48, A64, A68, A75, A77][B96–B99]. Such a challenge arises for three main reasons. First, the enormous plethora of frameworks that implement the same functionalities makes it hard to select the better one. Second, the lack of readable documentation for each framework ulteriorly hardens the decision process. Third, **bots are complex systems to design from an architectural point of view** [A3, A48, A72, A77][B100] because, to improve modularity, engineers push to adopt client-server architectures in which the core functionalities are implemented on the server and the interaction with the user on the client side. This, in addition to adding complexity due to the management of communication protocols between the various components (e.g., HTTPS, API Call, and Service Calls), often makes it necessary to use different technologies for the same system, and **not all bot frameworks allow easy**

integration with them [A48, A64, A68]. Furthermore, all the previously mentioned problems become harder when practitioners deal with AI-powered bots; in this type of bot, in addition to the integration of the already numerous technologies involved, it is necessary to integrate artificial intelligence systems in a way that meets the required nonfunctional requirements [A48].

Testing. Furthermore, despite such a large amount of frameworks, **a significant minority support developers in testing activities of bots and CAs** [A3, A26, A64, A76][B90]. This is a significant problem because the lack of testing for such a system contributes to the mistrust of practitioners when they have to use them. Underlying this lack is the difficulty in automating the interactions between a human user and the bot. Furthermore, the large number of frameworks for bot development makes it difficult to provide a single tool capable of testing for all.

Maintenance. Such a challenge in terms of testing leads to the emergence of another one: **the maintenance—and consequent evolution—of bots is challenging** [A1, A2, A26]. In fact, the lack of tests and the difficulty in monitoring such systems can impact maintenance activities, making it hard to identify and fix a bug.

Training. Training AI-powered bots for natural language understanding and processing is a complex endeavor, especially within the specific context of SE. Obtaining suitable datasets for training these tools is a significant challenge [A3, A39, A61, A68, A69]. Not only are these datasets scarce, but they are also particularly difficult to acquire when tailored to the nuanced requirements of SE [A48, A61]. This scarcity and specificity of data lead to **considerable difficulties in effectively training AI modules to perform their intended tasks**.

4.3 RQ3: On the Best Practices to Use Bots and Conversational Agents for Software Engineering Purposes

Besides collecting limitations and challenges for software bots, we were also interested in the possible best practices to overcome them. For such a purpose, we formulated our third research question to extract a list of best practices from the formal and GL to help practitioners adopt and develop bots for engineering purposes.

We decided to divide the identified best practices into two categories based on the approach's scope. In the following, we elaborated on each best practice individually.

4.3.1 Development and Design Best Practices. As already reported in Section 4.2.3, the large number of available technologies and the need for standardized methodologies make developing and designing bots challenging. In the following, we report the best practices identified by researchers and practitioners that can facilitate the implementation of bots and conversational systems (Appendix B—Table 1 [134]).

Follow a Modular Architecture. Despite their significant heterogeneity, bots—particularly CAs—are characterized by the need to work similarly on different platforms. For example, CAs should be used on different communication platforms—e.g., Slack and Discord—but still implement the same functionalities. In these situations, a client-service architecture—or derived ones—represents the best choice [A2, A65]. Srivastava and Prabhakar [A3] defined a reference architecture for CAs which specifies that these systems should be divided into two macro-components: (1) *Conversational Subsystem*, which implements the input analysis, the extraction of user intent, and the response; (2) *Business Component*, which contains the logic to fulfill user intents and is implemented like an API. Another possible architecture [B100] can organize chatbots into five modules:

- (1) *Environment* represents the core Natural Learning Process (NLP) engine and context interpretation.
- (2) *Question and Answer System* is where the system interprets the question and responds with relevant answers from the knowledge base.

- (3) *Plugins* is the module that uses API calls to interrogate other systems for executing bot operations.
- (4) *Node Server* is the module that handles the traffic requests from users and routes them to appropriate components.
- (5) *Front-end Systems* are the possible client-facing platforms users can use to interact with the system.

Make Bots Adaptive and Able to Learn Over Time Using Artificial Intelligence. Software bots, specifically in the figure of CAs, need to communicate with a large plethora of users characterized by different cultural and language characteristics. Therefore, in most cases, such interaction and the bot’s user understanding rely on an NLP module—that uses datasets composed of possible phrases and assertions in different languages—to interpret the user’s intent. However, the dataset employed should increase over time based on the user’s usage. For this reason, implementing a machine learning system able to learn over time and modify the dataset according to new conditions is needed—namely, *active learning systems* [149]—becomes necessary [B85, B89, B90, B94].

Adopt a Specific Lifecycle Process for Bots. Bots are complex systems to design, both from a technical and logical side, especially if integrated with a conversational module, i.e., like for CAs. Consequently, practitioners focused their attention on defining rigorous steps to orchestrate these systems from a development and maintenance perspective [B101–B103]. In the following, we reported the elicited steps for CAs development:

- (1) clearly defines the process to be automated or supported by the CA;
- (2) catch the target audience’s needs by engaging with software developers through interviews and usability testing sessions to identify common challenges they face during the development cycle;
- (3) use existing sources to analyze competitors by conducting surveys of similar tools already existing or interviewing the potential audience;
- (4) define audience intents using use cases and journey maps after conducting individual interviews with the audience;
- (5) design the interaction workflow for each intent by using modeling languages, e.g., UML;
- (6) choose technologies for the CA development and operation platforms according to company needs;
- (7) design the CA architecture to be modular as much as possible and adherent to the guidelines proposed by researchers [A2, A3, A65][B100];
- (8) develop a measurement framework to evaluate the CA by establishing metrics such as reduction in time to fix bugs, frequency of use, and developer satisfaction to assess the impact of the CA on the development process [A35, A49, A78][B85, B100];
- (9) implement an artificial intelligence (NLP, NLU, and active learning) module capable of understanding technical language specific to software development, using, for example, libraries that can parse and make sense of code snippets and technical queries [A61];
- (10) implement the CA application and interaction logic in a way that can handle diverse inputs from developers, such as textual descriptions, code blocks, or even spoken commands, and provide appropriate responses or actions [A35, A54, A60, A67, A79, A135][B85, B101–B103];
- (11) deploy the CA and adopt a continuous improvement process.

Carefully Select the Correct Framework Based on Bot Requirements. As mentioned in Section 4.2.3, the increasing adoption of bots and CAs in software development leads diverse actors to develop frameworks for supporting their development. Some examples are Azure Bot from Microsoft—able to provide a single environment to develop bots for different communication channels—or Rasa—an open-source solution to develop bots and train NLP models all in the same place. Nevertheless, determining which framework best fits a project’s requirements is far from

easy [A1, A64, A68, A69, A75][B96–B99]. To address this challenge, practitioners identified criteria and key motivations that should guide while selecting the development framework, i.e., core technology, the target platform of operation, and the distinctive characteristics [B96–B99]. We reported the main framework with associated motivations in Appendix B—Table 2 [134].

Adopt Domain Specific Language Models to Develop Bots. Developing bots requires expertise in several areas, such as SE and natural language processing. Although frameworks and platforms simplify the implementation of CA, there is still a significant obstacle caused by the inability of such frameworks to support all the developers’ desires. Some studies suggest approaches based on domain-specific languages (DSLs) for model-driven development, reducing the workload of developers and designers [A77]. Beyond that, such studies indicate that there are already bots developed to support the developer in building models that can then be used to develop systems—e.g., recommendation systems—in an automated manner [A11].

Make the Bot Able to Save and Use Contextual Information Using RAG and Vector DBs. Maintaining contextual information is critical for adopting bots in SE [A1, A19, A67, A70][B85, B90]. Integrating Retrieval-Augmented Generation (RAG) models and Vector Databases can address this challenge [A71, A79][B93]. RAG models combine retrieval and generation capabilities to access and use context-specific information effectively. Vector Databases store and query contextual data embeddings, enabling precise and context-aware responses. This synergy ensures chatbots provide more meaningful and context-rich conversations, enhancing the overall user experience.

Conduct Wizard of the Oz Studies to Create Datasets for NL Modules. Wizard of Oz studies create datasets for training NLP and NLU models for developer-support bots [A61]. Human operators simulate AI responses, allowing researchers to collect nuanced language data from developers. This method captures realistic insights into the support developers need and the queries they pose. Despite the complexity of these studies, collecting extensive data is essential for developing effective models [B94].

4.3.2 Interaction and Adoption Best Practices. Interaction for bots is an important—if not the most important—aspect to take care of during their development and design. For such a reason, it is not surprising that the state of the art focused more on best practices to improve how bots communicate with users (Appendix B—Table 3 [134]).

Integrate Bots Actions in a Way that Does Not Interrupt Developer Workflow. Bots are naturally designed to automate some processes—or part of them—to allow users to waste less time and focus on other tasks [151]. Such automation comes with no problems: first, bots’ actions should be secure to allow users to trust them; second, bots should not interrupt users in their operation, or the advantage of the automation loses its value and usefulness [A1, A5, A25, A43, A45][B84, B85]. For example, many users found tool-recommender-bot interrupted existing processes—most notably by breaking software builds—and were discouraged from merging pull requests from our system [A54]. For the reasons mentioned above, software bots should be designed so that the results of their actions should not slow down or hinder the user’s actions, at least not unexpectedly and covertly [A25, A30, A35, A54, A60][B85, B101–B103]. In recent years, orchestrator bots, i.e., bots designed to manage and guide the actions of other bots, have emerged as an efficient way to integrate a large number of bots easily without interrupting the practitioner’s workflow [A43, A45].

Allow Developers to Edit Bot Configuration Easily. As mentioned in Section 4.2.2, configuring bots to allow their operation in different contexts is a crucial challenge [A1, A4]—both for their adoption and development. Therefore, developing the system providing users with easy ways to configure it—e.g., editable configuration files, graphical

interfaces for configuration, and exhaustive documentation—could be extremely useful in increasing bot adoption [A4, A16, A25, A30, A35]. Moreover, Erlenhov et al. [A25] demonstrated that such a best practice could mitigate the *Interruption and Noise problem* [A1]—elaborated in Section 4.2.1—through a design or configuration that is mindful of whom to notify of which events.

Provide Bots With Personality. User perception is essential for bots. Various studies [A5, A60, A135] and GL [B101–B103] reported that this perception can be highly influenced by the bot’s language register—as well as by its name and graphical elements. Specifically, ensuring that the bot is recognizable and distinguishable, i.e., providing it with a distinctive *personality*, can help in its adoption and use [A60, A135]. Therefore, the bot’s language should be casual, accessible, and fun; in a way that is not dull but friendly. Nevertheless, it should be paid attention to not trigger the *uncanny valley* effect, i.e., the phenomenon whereby a computer bears a near-identical resemblance to a human being and evokes a sense of unease or aversion in the person interacting with it [130, 141].

Design a Smooth and Frictionless Interaction. Various works identified and analyzed the phenomena of “noise”—i.e., annoying bot behaviors that lead to deterioration of communication in teams [157]. Consequently, designing a smooth and frictionless interaction becomes mandatory, and this could be achieved by carefully planning conversational flows, especially for conversational bots. This can be achieved by (1) allowing bots to detect dead ends in conversations and prompt users by giving hints on how to continue the interaction and (2) implementing mechanisms to provide continuous feedback to the user that the bot is listening [A35, A54, A60, A67, A135][B85, B101–B103]. Moreover, designing bots to be proactive—maybe integrating Generative AI in them—is a successful way to please practitioners and make their interactions smooth [A79]. However, this can be achievable if the natural language comprehension of the bot is well enough designed and developed [A67]. Moreover, designers should evaluate the use of graphical user interface elements—e.g., cards, buttons, and boxes—to facilitate the interaction and make the conversation more efficient [A135].

Enforce Transparency in Bots Actions and Outputs. Trust is a vital aspect of adopting bots for software development since such systems are often designed to automate product development and release operations, which are considered vital to the success of a software project. The first way to improve it is related to the *uncanny valley* effect: when users interact with a machine that seems too human to them, they begin feeling a sense of unease and revulsion [130, 141]. To avoid such a situation, developers should ensure that the bot’s software nature is always apparent and that situations perceived as ambiguous by the user are never created [A5, A30, A35, A135][B85]. Moreover, it should always be evident when the user is asked to take action and why the request is made. For example, implementing graphical elements that are activated only at the moment when the user has to give input can be an excellent way to request an action overt [A5, A135][B85].

Integrating Concepts From Behavioral Science in Interaction Flow. As already said, *recommendation bots*—namely, bots designed to provide user tips and recommendations—are largely adopted for software development purposes. Nevertheless, such bots’ recommendations seem to be ineffective if the interaction with the users violates social contexts within SE and interrupts the development workflow of programmers. For This reason, Brown and Parning [A72] proposed adopting behavioral science concepts to design and guide the interaction flow of conversational systems—e.g., *nudging* [154] and *choice architecture* [155]. Notably, elaborated in their work, they use 11 practical tools—provided by Johnson et al. [127]—to integrate *choice architecture* [155]—the way options are organized and presented to humans—in

recommendation bots and consequentially improve the recommendation process. From a practical point of view, bots should be designed according to three interaction principles:

- **Actionability.** It refers to the ease with which users can act on recommendations. To improve developers' perception of recommendations, better choices should be easier to take and implement.
- **Feedback.** It refers to the way pieces of information are provided to the user. To improve developers' perception of recommendations, useful and essential information should be provided to the users.
- **Locality.** It refers to the setting—both physical and temporal—surrounding the context of developer recommendations. To improve developers' perception of recommendations, they should be presented during their working hours and when they are in the workplace.

Moreover, a study performed by Robe et al. [A60] demonstrated that practitioners are more inclined to adopt and use bots that demonstrate adaptability, motivation, and social presence skills; such characteristics support developers' trust.

Provide Users With a High Level of Control Over Bots. Another way to improve interaction is through *nudges*—i.e., any aspect of the choice architecture that alters people's behavior in a predictable way without forbidding any options or significantly changing their economic incentives [154]. As a way to use nudge for conversational systems, Brown et al. [A54] reported that making easy and cheap recommendations—instead of forcing updates and changes to developers—could improve the users' perception of recommendation bots [A16, A19, A25, A35, A54, A67][B85]. For example, in the context of a bot that analyzes the source code to find quality problems, making comments above the incriminated line of code instead of automatically refactoring it could be a better choice. Another way to provide a sense of control to the user—as proposed and reported by Erlenhov et al. [A25]—is to make the bot ask for confirmation before starting to work on a task or at the end of interactions—in a *transaction-fashion*—to avoid misunderstanding.

Create a Reliable Test Infrastructure. An alternative way to make developers trust a development bot is to provide it with a reliable and public test infrastructure [A25][B90]. In such a way, the potential audience of the tool can evaluate it in advance and feel more comfortable with it. Nevertheless, testing bots can be challenging due to the need to simulate all possible user interactions [A3, A26, A64, A76][B90]. In order to facilitate such activity, practitioners propose to (1) use one of the provided frameworks for bots—and chatbots—testing—e.g., testYourBot¹¹ and Dimon¹²—and (2) develop and perform manual testing of bots based on their interaction flow [B90].

Integrate a Feedback System Allowing Users to Evaluate Bot Choices. Most problems that characterize software bots regard how systems interact with users. In most cases, these issues could be fixed by asking users what they want and what the specific problem is; therefore, implementing a system for practitioners to provide feedback and evaluate bots' actions is highly recommended. Although an automatic approach based on heuristics would be the most appropriate solution—but not the easiest one to implement—in which the user is directly asked to provide opinions can also reach the goal [A35, A49, A78][B85, B100].

Adapt the Bots to the Privacy Policies of Organizations. To enhance trust in bots using generative AI, developers should implement adapters at the bot interfaces. These adapters control the bot's access to information and responses, aligning operations with privacy policies [B94] and ensuring confidentiality and user consent. Configured to limit

¹¹testYourBot site: <https://shankar96.github.io/testYourBot/>

¹²Dimon site: <http://dimon.co/>

sensitive data collection, adapters reduce data breach risks and support data minimization [B94]. This integration enhances compliance with privacy regulations and bolsters trust in the bot’s application.

4.4 RQ4: On the Benefits of Bots and Conversational Agents for Software Engineering Purposes

In this RQ, we aimed to identify the benefits for development teams of adopting bots for SE purposes. Specifically, our goal was to determine which areas are most impacted by bots to identify possible fields yet to be explored and propose future steps for research to deepen (Appendix B—Table 4 [134]).

4.4.1 Productivity Benefits. Most of the benefits provided by bots regard the productivity aspects of software development. This is because bots have been designed initially **to automate processes considered unimportant and tedious for practitioners** [A8, A27][B104–B106]. For this reason, it does not look surprising that the most positive impact of their adoption—recorded by researchers and practitioners—is related to the productivity.

Furthermore, bots can **help developers complete tasks related to meaningful goals for the project** [A6, A8, A22, A35, A40, A42, A45, A135][B87, B88]. Such a benefit is defined by Storey et al. [A22] under the title of *effectiveness* and can be achieved in various ways. For example, by automatically capturing data and disseminating them to the entire team, bots can increase the developers’ awareness of the project situation [A22, A135]. An ulterior way to improve developers’ effectiveness is through conversation moderation—i.e., providing information during meetings or bringing the discussion back into focus when participants tend to digress [A6].

Bots can also impact productivity by **helping developers complete tasks more quickly** [A6, A8, A13, A15, A19, A22, A27, A35, A42, A44, A45, A47, A51, A52, A57][B86, B104, B105]—this is identified by *efficiency* [A22]. Such a benefit can be reached in many ways: (1) bots can spare developers from reading large amounts of documentation by understanding what they search for and providing it immediately [A22, A52]; (2) bots can support during source code writing by providing code snippets or tips [A15, A51][B105]; (3) bots can also help in documentation phases, by collecting pieces of information from different sources and summary them [A6][B104].

In practice, bots are also used to perform simple tasks, e.g., building a new version of a software project, executing test suites, and preparing execution reports, thus implying an increase in workers’ free time and optimizing the execution of these tasks. This is because bots **can work without interruption** [A25][B82, B87, B88, B105] and **handle large amounts of tasks simultaneously** [A25][B86–B88, B104, B105].

4.4.2 Collaboration Benefits. Bots also provide benefits when comes to knowledge sharing and collaboration in development teams [A6, A8, A37, A44, A46, A135][B86, B87, B104]. For example, if correctly used, bots can distribute to the team the status of other tasks, consequently **building team trust and cooperation** [A135]. Furthermore, bots can enhance the recording of decisions, performing: (1) automatic fulfillment of artifacts templates (e.g., user stories), (2) summary build from discussions, (3) automatic generation of diagrams, and (4) monitoring of conversations to automatically refer previous decisions. Moreover, bots can enhance developer collaboration by performing (1) recording and retrieving technical discussions, (2) fetching and updating models based on conversation, and (3) parsing committed messages. By doing all of these, bots can **support the decision-making process** of different figures involved in SE [A43][B86].

Furthermore, by abstracting practitioners from executing repetitive and tedious tasks, there is **less chance of mistakes** [B104–B107]. This is since (1) humans tend to commit errors if constantly repeating the same task, and (2) software can apply quality standards pre-defined.

Moreover, such an abstraction can be used by managers to improve the risk management activity [B104, B105, B107] and **facilitate the adoption of new methodologies and technologies** in their team [B104, B106]. For example, by using a chatbot to improve code review, managers could provide developers with easy tips and good practices without assigning human effort to such a purpose.

4.4.3 Technical Benefits. Lastly, bots and CAs can also provide many benefits from a technical and product view. For example, bots can **improve the maintainability and quality of source code** by detecting technical debt—e.g., code smells and test smells—or by enforcing good programming practices [A8, A15–A19, A25, A27, A51][B82, B104–B107]. Moreover, bots can collect information from different artifacts—e.g., source code, documentation, and reports—to spread to the entire team and **reduce socio-technical problems**, e.g., turnover. Furthermore—it is always related to the knowledge-sharing side—bots can also be used to collect information about API and new technologies and provide them “on demand” to the users. Such communication can occur in any communication channel used by the development team [A8, A25, A31, A50, A56, A57, A135][B82, B86, B106].

5 DISCUSSION, IMPLICATIONS, AND FUTURE STEPS

The results of our study pointed out several observations worth further discussion. We started from them to identify potential future research lines that the research community can explore. Moreover, since the MLR also comprises GL, practitioners can benefit from our observations. In the following paragraphs, we report the observations and implications from our review, organized by topic.

5.1 Conversational Agents and Human Aspects of Software Development

CAs are *de facto* software systems in which interaction with users represents the main characteristic, thus implying that making the potential audience more involved during design phases is crucial to project success [A1, A3, A19, A25, A39, A63, A64, A66–A68, A135][B85, B89–B92]. Moreover, since most of the reported software bots operate with different stakeholders simultaneously, the interaction patterns between different actors also acquire importance. For this reason, representing collaboration and communication patterns in software communities in a quantitative way could help facilitate the CAs’ interaction workflow design. In order to operationalize such collaboration patterns, state of the art defined *community smells* [153]—i.e., a set of anti-patterns in collaboration activities in software development communities that are precursors of socio-technical problems. Based on the above consideration, we can provide two future fields of exploration:

- ✎ Community Smells to support bots design. Community Smells could be used as input when designing bots to improve communication and increase the trust that users have in them. Researchers could study how this aspect impacts the development process.
- ✎ Bots-Community Smells. Community Smells represent collaboration and communication anti-patterns in software communities. Adopting bots for software development purposes could lead to the creation of a community over the community, characterized by the interaction between bots and developers and bots with other bots. This could lead to the emergence of a new way to represent patterns—and anti-patterns—through a particular type of community smells. The research community could (1) quantitatively represent them and (2) provide practitioners with instruments to improve their relationship with conversational systems.

5.2 Conversational Agents for Training and Support

Besides practical uses, the adoption of bots also revolves around knowledge sharing [A22, A24, A25, A31, A41, A49, A50, A52, A53, A56–A58][B82, B85, B87, B88]. Specifically, practitioners use bots to (1) share information between team members and (2) record decision rationale. Moreover, CAs are used to provide information on software modules to new developers in order to facilitate training phases. Based on our findings, we can provide the following considerations:

- ✚ Bots and Turnover. Bots can be used to collect expert knowledge and provide it to newcomers [B82, B85]. In this way, the consequences of turnover [117]—i.e., the continuous influx and retreat of human resources in a team—can mitigate the progressive loss of knowledge on a software module. From here, researchers should focus on how (i) bots should collect such knowledge and (ii) how this should be provided to newcomers.

5.3 Addressing Limitations with Best Practices

Our literature survey identified challenges and best practices related to bots and CAs [157]. We tried to map such challenges to best practices based on the various studies to provide solutions and mitigation strategies supported by evidence in research and practitioners' contexts. We reported our findings in Figure 3.

- ✚ Challenges and Best Practices. Our mapping—based on literature evidence—should be validated through qualitative studies and could be a valuable contribution for practitioners facing bot challenges.

5.4 Conversational Agents for Project Management: How far are we?

Project Management is the practice of applying knowledge, skills, methods, and tools to meet the requirements of a specific project, achieving quality and performance goals [125]. In the context of software development, project management is a highly complex and hard-to-standardize activity due to the volatile nature of software and the extreme heterogeneity of stakeholders involved—both in terms of background and competencies [112, 113, 123]. Moreover, according to the Standish Group's CHAOS Report [128], approximately 70% of software projects fail—some of them with catastrophic consequences [118, 124, 129] and state of the art demonstrated that management aspects significantly impact project outcomes, leading management activities as a priority to consider [110, 114]. Software bots—and particularly CAs—could help improve and support management activities:

- ✚ CAs as Managers' Twins. Project managers (PM) are responsible for ensuring that the project meets the requirements. In order to achieve this objective, managers should have various skills and—most critically—manage various factors simultaneously [125]. To support this goal, having a community of bots collecting different metrics and monitoring different activities could support PMs in their daily tasks [A34]. Moreover, making a hierarchical bots structure in which pieces of information are filtered and summarized differently at each level of the organization could help managers focus on essential tasks, resulting in increased productivity and mental health quality [A43].
- ✚ CAs as Meeting Facilitator. There are tools designed to help practitioners manage information sharing during meetings [A6, A8, A37][B85]. Since SE is *de facto* a social activity involving different stakeholders with different backgrounds, having such support could greatly benefit the state of practice. However, the reported effectiveness of these tools could be better. Therefore, investing in improving their capabilities and using Generative AI could be a worthy future research topic.

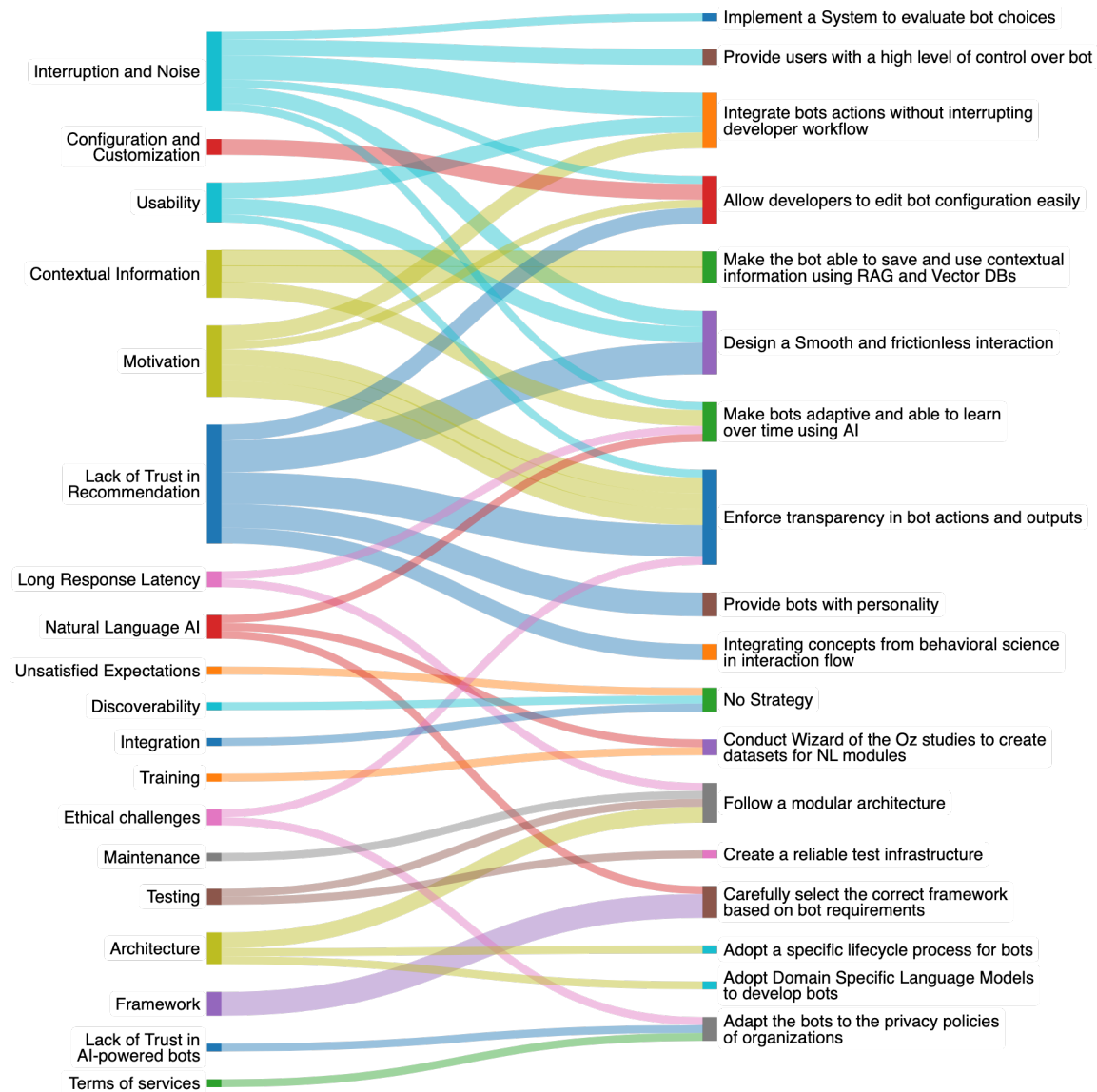


Fig. 3. Sankey diagram showing the identified challenges in bot development/adopton and their corresponding potential solutions. Challenges on the left, such as usability, trust, and customization, are linked to solutions on the right, including enhancing control, transparency, and AI adaptability.

5.5 Open Issues for Bots and Conversational Agents

Despite the significant attention reserved to the topic by researchers and practitioners, using bots for software development is still a young and unexplored field. For such a reason, there are a series of open issues and exploration opportunities that the research community should care of:

- ✚ Usability Research for CAs. Although CAs put the interaction with users at their center, to the best of our knowledge, there needs to be more research on the usability of such systems. Moreover, practitioners should put more effort into defining standards and guidelines to design interaction workflows in conversational systems.
- ✚ Testing of Bots. Testing bots and CAs arose as a complex topic [A3, A26, A64, A76][B90]. Most of the testing strategies for these systems fall under the category of system testing, and only a few have been proposed at lower levels of testing (e.g., unit and integration).
- ✚ Behavioral Science and Bots. Motivating users to adopt bots is a widely documented problem [A1, A5, A25][B84, B85, B92]. Various works proposed adopting behavioral science concepts—e.g., nudges and interaction principles—to improve the design phases of these systems [A72]. For this aim, researchers from both fields—computer science and social science—could join forces to open new research opportunities.
- ✚ Interruption and Noise: an Open Challenge. Interruption and noise are significant and documented problems for CAs and bots [A1, A25, A63, A135][B85]. Despite the work already done to propose mitigation strategies for such phenomena, more should be done to increase the adoption and effectiveness—in terms of accomplished contribution—of bots for development purposes.
- ✚ Generative AI for Software Development. The recent emergence of large language models in the consumer market has inevitably led software engineers to experiment with integrating these tools into their workflow [A9, A13]. When such LLMs are used in their chatbot versions (e.g., ChatGPT), they constitute software bots. Despite their impressive capabilities and the potential they could have in all aspects of software development, some clear limitations continue to exist, likely due to the technology’s still early stage.

6 LIMITATIONS AND THREATS

Threats to the validity of an MLR review are typically related to the inaccuracy of data extraction and incomplete set of studies due to search terms limitation, selection of academic search engines, and researcher bias regarding exclusion/inclusion criteria. In this section, we identified and carefully addressed potential threats to the validity [132, 158] of our study by taking steps to minimize or mitigate them.

Internal Validity. Threats in this category regard the source and data selection approach, as well as the possibility that the findings are affected by casualty factors without the researcher’s knowledge [158]. We relied on a previous systematic approach proposed in the literature [132] and described in Section 3. We defined and reported each step of the process carefully—i.e., search engines, search terms, and inclusion/exclusion criteria—to ensure replicability. These are usually the problematic steps. As for the search terms used for studies extraction, we relied not only on the most common words—e.g., bot—but also on similar terms used by different authors for pointing to a similar concept—e.g., CAs. We followed a formal search using defined keywords, followed by a manual search using the references of the initial pool in our field of study. As for the search engine, we included official comprehensive academic databases, e.g., Scopus, but also fewer ones like Google Scholar. While for the GL, we relied on Google Search Engine—like other GL and MLRs [122, 126]. Therefore, we believe that we collected all the relevant studies in the field. Finally, as for the inclusion/exclusion criteria, they can usually suffer from researchers’ judgment and experience. For this reason, all the paper’s authors were involved in checking criteria by applying joint voting for study selection. As a result, only those with high scores were selected for this study.

Construct Validity. Threats in this category regard the extent to which the object of study truly represents the theory behind the study and are mainly due to imprecision in performed measurements [158]. One threat in this

category concerns the lack of empirical evidence in the studies coming from GL. Indeed, this literature reports facts based on opinion or experience. Since the GL represents the voice of the practitioners, we could assume that they would be speaking from personal experience. This can be a crucial limitation; however, the goal of an MLR is also to show different ways of thinking, i.e., empirical versus practical, since it can have two types of audience: academic and industrial. Moreover, the inclusion of GL can help to overcome the scarcity of studies retrieved by automatic searches in the digital databases of scientific literature.

Conclusion Validity. Threats in this category concern the degree to which the conclusion we reach is credible or believable [158]. The first two authors of the paper reviewed each step of the process to ensure the reliability of the study and mitigate bias in data selection and extraction. Each severe disagreement between authors was resolved by consensus among all the authors. Furthermore, in order to strengthen our results, we followed both the guidelines from Kitchenham et al. [131]—related to the formal literature selection—and from Garousi et al. [121]—associated with the GL inclusion. Moreover, we followed a rigid quality criteria assessment—both for WL and GL—and we extracted our sources from the most used and known databases.

External Validity. Threats in this category concern the extent to which our study’s results can be generalized [158]. First of all, we conducted our MLR on the most known and suggested databases of literature [121, 131] to increase the quality of extracted sources and generalizability of our findings. Furthermore, we extracted only primary sources written in English, so the rest in other languages were excluded, thus possibly threatening our generalizability. Nonetheless, (i) the top venues in SE accept only studies written in English, and (ii) most of the literature reviews already present in the literature include exclusion criteria based on the English language.

7 CONCLUSION

In this work, we reported on an MLR to shed light on adopting bots and CAs for SE purposes. The reason behind the choice of an MLR is related to the fact that bots and CAs are extremely linked to the practitioners’ field, so we wanted to contribute both to researchers and practitioners. Specifically, we aimed to provide a representation and categorization of the (1) motivation for adopting bots, (2) challenges arising from their integration into software development processes, (3) best practices applicable contrasting the before-mentioned challenges, and (4) benefits derived from the adoption of bots. Moreover, we also provide a list of discussion points and opportunities for researchers to encourage new investigations on the matter. Indeed, our contributions could be considered a fundamental base built to advance the state of the art and provide an advantage both for researchers and practitioners.

ACKNOWLEDGMENTS

The authors would like to thank the two anonymous Reviewers and the editor for their insightful comments on our paper.

MLR—FORMAL LITERATURE

- [A1] Mairieli Wessel, Igor Wiese, Igor Steinmacher, and Marco Aurelio Gerosa. Don’t disturb me: Challenges of interacting with software bots on open source software projects. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW2):1–21, 2021.
- [A2] Shayan Zamanirad, Boualem Benatallah, Moshe Chai Barukh, Fabio Casati, and Carlos Rodriguez. Programming bots by synthesizing natural language expressions into api invocations. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 832–837, 2017.

- [A3] Saurabh Srivastava and TV Prabhakar. A reference architecture for applications with conversational components. In *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, pages 1–5. IEEE, 2019.
- [A4] Rijnard van Tonder and Claire Le Goues. Towards s/engineer/bot: Principles for program repair bots. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 43–47. IEEE, 2019.
- [A5] Martin Monperrus. Explainable software bot contributions: Case study of automated bug fixes. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 12–15, May 2019.
- [A6] Fabian Gilson and Danny Weyns. When natural language processing jumps into collaborative software engineering. In *2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*, pages 238–241, March 2019.
- [A7] Ranci Ren, John W. Castro, Adrián Santos, Sara Pérez-Soler, Silvia T. Acuña, and Juan de Lara. Collaborative modelling: Chatbots or on-line tools? an experimental study. In *Proceedings of the Evaluation and Assessment in Software Engineering, EASE '20*, page 260–269, New York, NY, USA, 2020. Association for Computing Machinery.
- [A8] Fabian Gilson, Sam Annand, and Jack Steel. Recording software design decisions on the fly. In *SEED/NLPaSE@ APSEC*, pages 53–66, 2020.
- [A9] Justine Winata Purwoko, Tegar Abdullah, Budiman Wijaya, Alexander Agung Santoso Gunawan, and Karen Etania Saputra. Analysis chatgpt potential: Transforming software development with ai chat bots. In *2023 International Conference on Networking, Electrical Engineering, Computer Science, and Technology (IConNECT)*, pages 36–41. IEEE, 2023.
- [A10] Rijul Saini, Gunter Mussbacher, Jin LC Guo, and Jörg Kienzle. Automated, interactive, and traceable domain modelling empowered by artificial intelligence. *Software and Systems Modeling*, 21(3):1015–1045, 2022.
- [A11] Lissette Almonte, Sara Pérez-Soler, Esther Guerra, Iván Cantador, and Juan de Lara. Automating the synthesis of recommender systems for modelling languages. In *Proceedings of the 14th ACM SIGPLAN International Conference on Software Language Engineering*, pages 22–35, 2021.
- [A12] Ilham Qasse, Shailesh Mishra, Björn þór Jónsson, Foutse Khomh, and Mohammad Hamdaqa. Chat2code: A chatbot for model specification and code generation, the case of smart contracts. In *2023 IEEE International Conference on Software Services Engineering (SSE)*, pages 50–60. IEEE, 2023.
- [A13] Gian Luca Scoccia. Exploring early adopters’ perceptions of chatgpt as a code generation tool. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, pages 88–93. IEEE, 2023.
- [A14] Sara Pérez-Soler, Esther Guerra, and Juan de Lara. Flexible modelling using conversational agents. In *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pages 478–482, Sep. 2019.
- [A15] Marvin Wyrich and Justus Bogner. Towards an autonomous bot for automatic source code refactoring. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 24–28. IEEE, 2019.
- [A16] Vahid Alizadeh, Mohamed Amine Ouali, Marouane Kessentini, and Meriem Chater. Refbot: intelligent software refactoring bot. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 823–834. IEEE, 2019.
- [A17] Zhewei Hu and Edward F. Gehring. Improving feedback on github pull requests: A bots approach. In *2019 IEEE Frontiers in Education Conference (FIE)*, pages 1–9, Oct 2019.
- [A18] Antônio Carvalho, Welder Luz, Diego Marcílio, Rodrigo Bonifácio, Gustavo Pinto, and Edna Dias Canedo. C-3pr: A bot for fixing static analysis violations via pull requests. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 161–171, Feb 2020.
- [A19] Marvin Wyrich., Regina Hebig., Stefan Wagner., and Riccardo Scandariato. Perception and acceptance of an autonomous refactoring bot. In *Proceedings of the 12th International Conference on Agents and Artificial Intelligence - Volume 1: ICAART.*, pages 303–310. INSTICC, SciTePress, 2020.
- [A20] Marvin Wyrich, Raoul Ghit, Tobias Haller, and Christian Müller. Bots don’t mind waiting, do they? comparing the interaction with automatically and manually created pull requests. *arXiv preprint arXiv:2103.03591*, 2021.
- [A21] Dragos Serban, Bart Golsteijn, Ralph Holdorp, and Alexander Serebrenik. Saw-bot: Proposing fixes for static analysis warnings with github suggestions. In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*, pages 26–30, June 2021.
- [A22] Margaret-Anne Storey and Alexey Zagalsky. Disrupting developer productivity one bot at a time. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016*, page 928–931, New York, NY, USA, 2016. Association for Computing Machinery.
- [A23] Simon Urli, Zhongxing Yu, Lionel Seinturier, and Martin Monperrus. How to design a program repair bot? insights from the repairnator project. In *2018 IEEE/ACM 40th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, pages 95–104. IEEE, 2018.
- [A24] Elahe Paikari and André van der Hoek. A framework for understanding chatbots and their future. In *Proceedings of the 11th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE '18*, page 13–16, New York, NY, USA, 2018. Association for Computing Machinery.
- [A25] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. An empirical study of bots in software development: Characteristics and challenges from a practitioner’s perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, page 445–455, New York, NY, USA, 2020. Association for Computing Machinery.
- [A26] Linda Erlenhov, Francisco Gomes de Oliveira Neto, Martin Chukaleski, and Samer Daknache. Challenges and guidelines on designing test cases for test bots. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW’20*, page 41–45, New York, NY, USA, 2020. Association for Computing Machinery.

- [A27] Dušan Okanović, Samuel Beck, Lasse Merz, Christoph Zorn, Leonel Merino, André van Hoorn, and Fabian Beck. Can a chatbot support software engineers with load testing? approach and experiences. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, ICPE '20, page 120–129, New York, NY, USA, 2020. Association for Computing Machinery.
- [A28] Tapajit Dey, Sara Mousavi, Eduardo Ponce, Tanner Fry, Bogdan Vasilescu, Anna Filippova, and Audris Mockus. Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories*, MSR '20, page 209–219, New York, NY, USA, 2020. Association for Computing Machinery.
- [A29] Tapajit Dey, Bogdan Vasilescu, and Audris Mockus. An exploratory study of bot commits. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, ICSEW'20, page 61–65, New York, NY, USA, 2020. Association for Computing Machinery.
- [A30] Runzhi He, Hao He, Yuxia Zhang, and Minghui Zhou. Automating dependency updates in practice: An exploratory study on github dependabot. *IEEE Transactions on Software Engineering*, 49(8):4004–4022, 2023.
- [A31] Ivan Beschastnikh, Mircea F. Lungu, and Yanyan Zhuang. Accelerating software engineering research adoption with analysis bots. In *2017 IEEE/ACM 39th International Conference on Software Engineering: New Ideas and Emerging Technologies Results Track (ICSE-NIER)*, pages 35–38, 2017.
- [A32] Mairieli Wessel, Igor Steinmacher, Igor Wiese, and Marco A Gerosa. Should i stale or should i close? an analysis of a bot that closes abandoned issues and pull requests. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 38–42. IEEE, 2019.
- [A33] Ehsan Mirsaeedi and Peter C. Rigby. Mitigating turnover with code review recommendation: Balancing expertise, workload, and knowledge distribution. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 1183–1195, New York, NY, USA, 2020. Association for Computing Machinery.
- [A34] Gianmario Voria, Viviana Pentangelo, Antonio Della Porta, Stefano Lambiase, Gemma Catolino, Fabio Palomba, and Filomena Ferrucci. Community smell detection and refactoring in slack: The cadocs project. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 469–473. IEEE, 2022.
- [A35] Zhendong Wang, Yi Wang, and David Redmiles. Optimizing workflow for elite developers: Perspectives on leveraging se bots. In *2023 IEEE/ACM 5th International Workshop on Bots in Software Engineering (BotSE)*, pages 23–27. IEEE, 2023.
- [A36] Abdelrahman H Hefny, Georgios A Dafoulas, and Manal A Ismail. A proactive management assistant chatbot for software engineering teams: Prototype and preliminary evaluation. In *2021 3rd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, pages 295–300. IEEE, 2021.
- [A37] Soomin Kim, Jinsu Eun, Changhoon Oh, Bongwon Suh, and Joonhwan Lee. Bot in the bunch: Facilitating group chat discussion by improving efficiency and participation with a chatbot. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, page 1–13, New York, NY, USA, 2020. Association for Computing Machinery.
- [A38] Elaha Paikari, JaeEun Choi, SeonKyu Kim, Sooyoung Baek, MyeongSoo Kim, SeungEon Lee, ChaeYeon Han, YoungJae Kim, KaHye Ahn, Chan Cheong, et al. A chatbot for conflict detection and resolution. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 29–33. IEEE, 2019.
- [A39] Jhonny Cerezo, Juraj Kubelka, Romain Robbes, and Alexandre Bergel. Building an expert recommender chatbot. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 59–63, May 2019.
- [A40] Arkadip Basu and Kunal Banerjee. Designing a bot for efficient distribution of service requests. In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*, pages 16–20, June 2021.
- [A41] Zeyu Ni, Beijun Shen, Yuting Chen, Zhangyuan Meng, and Junming Cao. Crowdevbot: A task-oriented conversational bot for software crowdsourcing platform (s). In *SEKE*, pages 410–522, 2019.
- [A42] Mairieli Wessel, Alexander Serebrenik, Igor Wiese, Igor Steinmacher, and Marco A. Gerosa. Effects of adopting code review bots on pull requests to oss projects. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 1–11, Sep. 2020.
- [A43] Mairieli Wessel, Ahmad Abdellatif, Igor Wiese, Tayana Conte, Emad Shihab, Marco A Gerosa, and Igor Steinmacher. Bots for pull requests: The good, the bad, and the promising. In *Proceedings of the 44th International Conference on Software Engineering*, pages 274–286, 2022.
- [A44] Mairieli Wessel, Alexander Serebrenik, Igor Wiese, Igor Steinmacher, and Marco A Gerosa. Quality gatekeepers: investigating the effects of code review bots on pull request activities. *Empirical Software Engineering*, 27(5):108, 2022.
- [A45] Eric Ribeiro, Ronan Nascimento, Igor Steinmacher, Laerte Xavier, Marco Gerosa, Hugo de Paula, and Mairieli Wessel. Together or apart? investigating a mediator bot to aggregate bot's comments on pull requests. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 434–438. IEEE, 2022.
- [A46] Christoph Matthies, Franziska Dobrigkeit, and Guenter Hesse. An additional set of (automated) eyes: Chatbots for agile retrospectives. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 34–37. IEEE, 2019.
- [A47] Yuxia Zhang, Zhiqing Qiu, Klaas-Jan Stol, Wenhui Zhu, Jiaxin Zhu, Yingchen Tian, and Hui Liu. Automatic commit message generation: A critical review and directions for future work. *IEEE Transactions on Software Engineering*, 2024.
- [A48] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. Towards human-bot collaborative software architecting with chatgpt. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pages 279–285, 2023.
- [A49] Rahul Kumar, Chetan Bansal, Chandra Maddila, Nitin Sharma, Shawn Martelock, and Ravi Bhargava. Building sankie: An ai platform for devops. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 48–53, May 2019.

- [A50] Chun-Ting Lin, Shang-Pin Ma, and Yu-Wen Huang. Msabot: A chatbot framework for assisting in the development and operation of microservice-based systems. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW'20*, page 36–40, New York, NY, USA, 2020. Association for Computing Machinery.
- [A51] Scott A Carr, Francesco Logozzo, and Mathias Payer. Automatic contract insertion with ccbot. *IEEE Transactions on Software Engineering*, 43(8):701–714, 2016.
- [A52] Yuan Tian, Ferdian Thung, Abhishek Sharma, and David Lo. Apibot: Question answering bot for api documentation. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 153–158, 2017.
- [A53] Katsunori Fukui, Tomoki Miyazaki, and Masao Ohira. Suggesting questions that match each user’s expertise in community question and answering services. In *2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pages 501–506, July 2019.
- [A54] Chris Brown and Chris Parnin. Sorry to bother you: Designing bots for effective recommendations. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, pages 54–58, May 2019.
- [A55] Chaiyakarn Khanan, Worawit Luewichana, Krissakorn Pruktharathikoon, Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, Morakot Choetkiertikul, Chaiyong Ragkhitwetsagul, and Thanwadee Sunetnanta. Jitbot: an explainable just-in-time defect prediction bot. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pages 1336–1339, 2020.
- [A56] Ricardo Romero, Esteban Parra, and Sonia Haiduc. Experiences building an answer bot for gitter. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW'20*, page 66–70, New York, NY, USA, 2020. Association for Computing Machinery.
- [A57] James Dominic, Charles Ritter, and Paige Rodeghero. Onboarding bot for newcomers to software engineering. In *Proceedings of the International Conference on Software and System Processes, ICSSP '20*, page 91–94, New York, NY, USA, 2020. Association for Computing Machinery.
- [A58] Preetha Chatterjee, Kostadin Damevski, and Lori Pollock. Automatic extraction of opinion-based q&a from online developer chats. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1260–1272. IEEE, 2021.
- [A59] Mahsa H Sadi and Eric Yu. Rapid: a knowledge-based assistant for designing web apis. *Requirements Engineering*, 26(2):185–236, 2021.
- [A60] Peter Robe and Sandeep Kaur Kuttal. Designing pairbuddy—a conversational agent for pair programming. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 29(4):1–44, 2022.
- [A61] Peter Robe, Sandeep K Kuttal, Jake AuBuchon, and Jacob Hart. Pair programming conversations with agents vs. developers: challenges and opportunities for se community. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 319–331, 2022.
- [A62] Carlene Lebeuf, Margaret-Anne Storey, and Alexey Zagalsky. Software bots. *IEEE Software*, 35(1):18–23, January 2018.
- [A63] Samaneh Saadat, Natalia Colmenares, and Gita Sukthankar. Do bots modify the workflow of github teams? In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*, pages 1–5, June 2021.
- [A64] André M. Pinheiro, Caio S. Rabello, Leonardo B. Furtado, Gustavo Pinto, and Cleidson R.B. de Souza. Expecting the unexpected: Distilling bot development, challenges, and motivations. In *2019 IEEE/ACM 12th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, pages 51–52, May 2019.
- [A65] Jinho Lee, Inseok Hwang, Thomas S. Hubregtsen, Anne E. Gattiker, and Christopher M. Durham. Accelerating conversational agents built with off-the-shelf modularized services. *IEEE Pervasive Computing*, 18(2):47–57, April 2019.
- [A66] John Wilmar Castro, Ranci Ren, Silvia T Acuña, and Juan de Lara. Usability of chatbots: A systematic mapping study, 2019.
- [A67] Glaucia Melo, Edith Law, Paulo Alencar, and Donald Cowan. Understanding user understanding: What do developers expect from a cognitive assistant? In *2020 IEEE International Conference on Big Data (Big Data)*, pages 3165–3172, Dec 2020.
- [A68] Ahmad Abdellatif, Diego Costa, Khaled Badran, Rabe Abdalkareem, and Emad Shihab. Challenges in chatbot development: A study of stack overflow posts. In *Proceedings of the 17th International Conference on Mining Software Repositories, MSR '20*, page 174–185, New York, NY, USA, 2020. Association for Computing Machinery.
- [A69] Ahmad Abdellatif, Khaled Badran, Diego Elias Costa, and Emad Shihab. A comparison of natural language understanding platforms for chatbots in software engineering. *IEEE Transactions on Software Engineering*, 48(8):3087–3102, 2021.
- [A70] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. Dependency management bots in open-source systems—prevalence and adoption. *PeerJ Computer Science*, 8:e849, 2022.
- [A71] Gabriele De Vito, Stefano Lambiasi, Fabio Palomba, and Filomena Ferrucci. Meet c4se: Your new collaborator for software engineering tasks. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 235–238. IEEE, 2023.
- [A72] Chris Brown and Chris Parnin. Sorry to bother you again: Developer recommendation choice architectures for designing effective bots. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW'20*, page 56–60, New York, NY, USA, 2020. Association for Computing Machinery.
- [A73] Anze Gao, Yang Zhang, Tao Wang, Sihao Chen, and Jinsheng Deng. How does bot affect developer’s sentiment: An empirical study on github issues and prs. In *2022 IEEE Smartworld, Ubiquitous Intelligence & Computing, Scalable Computing & Communications, Digital Twin, Privacy Computing, Metaverse, Autonomous & Trusted Vehicles (SmartWorld/UIC/ScalCom/DigitalTwin/PriComp/Meta)*, pages 1856–1861. IEEE, 2022.
- [A74] Anze Gao, Sihao Chen, Tao Wang, and Jinsheng Deng. Understanding the impact of bots on developers sentiment and project progress. In *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*, pages 93–96. IEEE, 2022.

- [A75] Sara Pérez-Soler, Esther Guerra, and Juan de Lara. Creating and migrating chatbots with conga. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 37–40. IEEE, 2021.
- [A76] Jordi Cabot, Loli Burgueño, Robert Clarisó, Gwendal Daniel, Jorge Perianez-Pascual, and Roberto Rodriguez-Echeverria. Testing challenges for nlp-intensive bots. In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*, pages 31–34, June 2021.
- [A77] Charaf Ouaddi, Lamy Benaddi, and Abdeslam Jakimi. Architecture, tools, and dsls for developing conversational agents: An overview. *Procedia Computer Science*, 231:293–298, 2024.
- [A78] Zhewei Hu and Edward Gehringer. Bot with interactions: Improving github pull-request feedback through two-way communication. In *2023 IEEE/ACM 5th International Workshop on Bots in Software Engineering (BotSE)*, pages 28–32. IEEE, 2023.
- [A79] Emily Judith Arteaga Garcia, João Felipe Nicolaci Pimentel, Zixuan Feng, Marco Gerosa, Igor Steinmacher, and Anita Sarma. How to support ml end-user programmers through a conversational agent. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–12, 2024.

MLR—GREY LITERATURE

- [B80] Friend or foe: Ai chatbots in software development. Available at <https://www.synopsys.com/blogs/software-security/ai-chatbots-in-software-development.html> (2024/04/14).
- [B81] Best applications of large language models. Available at <https://indatalabs.com/blog/large-language-model-apps> (2024/04/14).
- [B82] Jordi Cabot. Best bots to improve your software development process. Available at <https://livablesoftware.com/best-bots-software-development/> (2022/07/14).
- [B83] Matt Francis. 4 things you absolutely need to know about software bots. Available at <https://workingmouse.com.au/innovation/4-things-you-absolutely-need-to-know-about-software-bots/> (2022/07/14).
- [B84] Software bot explained. Available at https://everythingexplained.today/Software_bot/ (2022/07/16).
- [B85] Margaret-Anne Storey, Alexander Serebrenik, Carolyn Penstein Rosé, Thomas Zimmermann, and James D Herbsleb. BOTse: Bots in software engineering (dagstuhl seminar 19471). In *Dagstuhl Reports*, volume 9. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [B86] A complete guide on the role of chatbots in devops. Available at <https://www.invensislearning.com/blog/chatbots-in-devops/> (2024/04/14).
- [B87] The role of bots and AI in project management. Available at <https://blog.proofhub.com/the-role-of-bots-and-ai-in-project-management-87d29b78e304> (2022/07/14).
- [B88] Four ways AI & bots will change project management. Available at <https://blog.planview.com/ai-bots-project-management/> (2022/07/14).
- [B89] 6 smart solutions to combat chatbot development challenges. Available at <https://www.goodfirms.co/blog/chatbot-development-challenges> (2022/07/14).
- [B90] 4 biggest challenges in chatbot development and how to avoid them. Available at <https://insights.daffodilsw.com/blog/4-biggest-challenges-in-chatbot-development-and-how-to-avoid-them> (2022/07/14).
- [B91] Chatbots: When to use NLP & when to use NLU. Available at <https://cobusgreyling.medium.com/chatbots-when-to-use-nlp-and-when-to-use-nlu-8eba17c0a4bc> (2022/07/14).
- [B92] 7 chatbot development challenges. Available at <https://www.a3logics.com/blog/chatbot-development-challenges-you-cannot-ignore> (2022/07/14).
- [B93] What is retrieval-augmented generation, and what does it do for generative ai? Available at <https://github.blog/2024-04-04-what-is-retrieval-augmented-generation-and-what-does-it-do-for-generative-ai/> (2024/04/14).
- [B94] Ai chatbots: Understanding the benefits and limitations. Available at <https://synoptek.com/insights/it-blogs/data-insights/ai-chatbots-understanding-the-benefits-and-limitations/> (2024/04/14).
- [B95] What's next for ai in 2024. Available at <https://www.technologyreview.com/2024/01/04/1086046/whats-next-for-ai-in-2024/> (2024/04/14).
- [B96] 10 best AI chatbot development frameworks comparison. Available at <https://www.spaceo.ca/blog/top-ai-chatbot-frameworks/> (2022/07/14).
- [B97] Top chatbot development frameworks you should know – exploring paid solutions (part 1). Available at <https://thirdeyedata.io/top-chatbot-development-frameworks-you-should-know-exploring-paid-solutions-part-1/> (2022/07/14).
- [B98] Probot. Available at <https://github.com/probot/probot> (2022/07/14).
- [B99] A comparison of eight chatbot environments. Available at <https://cobusgreyling.medium.com/updated-a-comparison-of-eight-chatbot-environments-7f57d4e2dc09> (2022/07/14).
- [B100] Understanding the conversational chatbot architecture. Available at <https://blog.vsoftconsulting.com/blog/understanding-the-architecture-of-conversational-chatbot> (2022/07/14).
- [B101] Implement a successful bot strategy in 12 steps. Available at <https://www.thinkhdi.com/library/supportworld/2020/implement-successful-bot-strategy-12-steps.aspx> (2022/07/14).
- [B102] 10 steps to define your chatbot strategy. Available at <https://www.digiteum.com/10-steps-to-define-your-chatbot-strategy/> (2022/07/14).
- [B103] 8 best practices for bot development. Available at <https://marutitech.com/8-best-practices-bot-development/> (2022/07/14).
- [B104] Bots: What they are and why your agile software development team should use them. Available at <https://www.klipfolio.com/blog/bots-agile-software-development> (2022/07/14).
- [B105] 5 reasons developers are choosing codebots. Available at <https://workingmouse.com.au/app-development/5-reasons-developers-are-choosing-codebots/> (2022/07/14).

- [B106] Using microsoft azure bots & AI to automate DevSecOps. Available at <https://www.xgility.com/webinar-recording-using-microsoft-azure-bots-ai-to-automate-devsecops/> (2022/07/14).
- [B107] Why robots should format our code for us. Available at <https://www.freecodecamp.org/news/why-robots-should-format-our-code-159fd06d17f7> (2022/07/14).

REFERENCES

- [108] Ahmad Abdellatif, Khaled Badran, and Emad Shihab. 2022. A Repository of Research Articles on Software Bots. <http://papers.botse.org>
- [109] Richard J Adams, Palie Smart, and Anne Sigismund Huff. 2017. Shades of grey: guidelines for working with the grey literature in systematic reviews for management and organizational studies. *International Journal of Management Reviews* 19, 4 (2017), 432–454.
- [110] Taimour Al Neimat. 2005. Why IT projects fail. *The project perfect white paper collection* 8 (2005).
- [111] João Paulo Biazotto, Daniel Feitosa, Paris Avgeriou, and Elisa Yumi Nakagawa. 2024. Technical debt management automation: State of the art and future perspectives. *Information and Software Technology* 167 (2024).
- [112] Frederick P Brooks Jr. 1995. *The mythical man-month: essays on software engineering*. Pearson Education.
- [113] Cristina T Cerdeiral and Gleison Santos. 2019. Software project management in high maturity: A systematic literature mapping. *Journal of Systems and Software* 148 (2019), 56–87.
- [114] Narciso Cerpa and June M. Verner. 2009. Why Did Your Project Fail? *Commun. ACM* 52, 12 (Dec. 2009), 130–134.
- [115] Daniela S Cruzes and Tore Dybå. 2010. Synthesizing evidence in software engineering research. In *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–10.
- [116] Alvaro Fernández Del Carpio and Leonardo Bermón Angarita. 2023. Assistant Solutions in Software Engineering: A Systematic Literature Review. In *2023 IEEE 14th International Conference on Software Engineering and Service Science (ICSESS)*. IEEE, 93–100.
- [117] Matthieu Foucault, Marc Palyart, Xavier Blanc, Gail C Murphy, and Jean-Rémy Falleri. 2015. Impact of developer turnover on quality in open-source software. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 829–841.
- [118] Terry Frieden. 2005. Report: FBI wasted millions on 'Virtual Case File'. Retrieved in April 2021.
- [119] John Gargani and Stewart I Donaldson. 2011. What works for whom, where, why, for what, and when? Using evaluation evidence to take action in local contexts. *New Directions for Evaluation* 2011, 130 (2011), 17–30.
- [120] Vahid Garousi, Michael Felderer, and Tuna Hacaloğlu. 2017. Software test maturity assessment and test process improvement: A multivocal literature review. *Information and Software Technology* 85 (2017), 16–42.
- [121] Vahid Garousi, Michael Felderer, and Mika V Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106 (2019), 101–121.
- [122] Vahid Garousi and Mika V Mäntylä. 2016. When and what to automate in software testing? A multi-vocal literature review. *Information and Software Technology* 76 (2016), 92–117.
- [123] Faisal Hayat, Ammar Ur Rehman, Khawaja Sarhad Arif, Kanwal Wahab, and Muhammad Abbas. 2019. The influence of agile methodology (Scrum) on software project management. In *2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 145–149.
- [124] Azham Hussain and Emmanuel OC Mkpogio. 2016. Requirements: Towards an understanding on why software projects fail. In *AIP Conference Proceedings*, Vol. 1761. AIP Publishing LLC, 020046.
- [125] Project Management Institute. 2021. *A Guide to the Project Management Body of Knowledge* (7 ed.). 250 pages.
- [126] Chadni Islam, Muhammad Ali Babar, and Surya Nepal. 2019. A multi-vocal review of security orchestration. *ACM Computing Surveys (CSUR)* 52, 2 (2019), 1–45.
- [127] Eric J Johnson, Suzanne B Shu, Benedict GC Dellaert, Craig Fox, Daniel G Goldstein, Gerald Häubl, Richard P Larrick, John W Payne, Ellen Peters, David Schkade, et al. 2012. Beyond nudges: Tools of a choice architecture. *Marketing letters* 23, 2 (2012), 487–504.
- [128] Jim Johnson. 2020. *CHAOS 2020: Beyond Infinity*. The Standish Group International Boston, MA.
- [129] Magne Jørgensen and Kjetil Moløkken-Østfold. 2006. How large are software cost overruns? A review of the 1994 CHAOS report. *Information and Software Technology* 48, 4 (2006), 297–301.
- [130] Norri Kageki. 2012. An uncanny mind: Masahiro Mori on the uncanny valley and beyond. *IEEE spectrum* 12, 06 (2012), 2012.
- [131] Staffs Keele et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, Ver. 2.3 EBSE Technical Report. EBSE.
- [132] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering—a systematic literature review. *Information and software technology* 51, 1 (2009), 7–15.
- [133] Indika Kumara, Martín Garriga, Angel Urbano Romeu, Dario Di Nucci, Fabio Palomba, Damian Andrew Tamburri, and Willem-Jan van den Heuvel. 2021. The do's and don'ts of infrastructure code: A systematic gray literature review. *Information and Software Technology* 137 (2021), 106593.
- [134] Stefano Lambiase, Gemma Catolino, Fabio Palomba, and Filomena Ferrucci. 2024. Motivations, Challenges, Best Practices, and Benefits for Bots and Conversational Agents in Software Engineering: A Multivocal Literature Review—Online Appendix. <https://doi.org/10.6084/m9.figshare.27211989>
- [135] Carlene Lebeuf, Margaret-Anne Storey, and Alexey Zagalsky. 2018. Software Bots. *IEEE Software* 35, 1 (January 2018), 18–23.

- [136] Tom Lewandowski, Jasmin Delling, Christian Grotherr, and Tilo Böhmann. 2021. State-of-the-Art Analysis of Adopting AI-based Conversational Agents in Organizations: A Systematic Literature Review. In *25th Pacific Asia Conference on Information Systems, Dubai, UAE*. 1–14.
- [137] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov. 2014. An empirical analysis of flaky tests. In *ESEC/FSE 2014*. 643–653.
- [138] Quenby Mahood, Dwayne Van Eerd, and Emma Irvin. 2014. Searching for grey literature for systematic reviews: challenges and benefits. *Research synthesis methods* 5, 3 (2014), 221–234.
- [139] Mika V Mäntylä and Kari Smolander. 2016. Gamification of software testing-an mlr. In *International conference on product-focused software process improvement*. Springer, 611–614.
- [140] R Moguel-Sánchez, CS Sergio Martínez-Palacios, JO Ocharán-Hernández, X Limón, and AJ Sánchez-García. 2023. Bots in Software Development: A Systematic Literature Review and Thematic Analysis. *Programming and Computer Software* 49, 8 (2023), 712–734.
- [141] Masahiro Mori, Karl F MacDorman, and Norri Kageki. 2012. The uncanny valley. *IEEE Robotics & automation magazine* 19, 2 (2012), 98–100.
- [142] Quim Motger, Xavier Franch, and Jordi Marco. 2022. Software-based dialogue systems: survey, taxonomy, and challenges. *Comput. Surveys* 55, 5 (2022), 1–42.
- [143] Rodney T Ogawa and Betty Malen. 1991. Towards rigor in reviews of multivocal literatures: Applying the exploratory case study method. *Review of educational research* 61, 3 (1991), 265–286.
- [144] Chinedu Wilfred Okonkwo and Abejide Ade-Ibijola. 2021. Chatbots applications in education: A systematic review. *Computers and Education: Artificial Intelligence* 2 (2021), 100033.
- [145] Dong-Min Park, Seong-Soo Jeong, and Yeong-Seok Seo. 2022. Systematic Review on Chatbot Techniques and Applications. *Journal of Information Processing Systems* 18, 1 (2022), 26–47.
- [146] Michael Quinn Patton. 1991. Towards utility in reviews of multivocal literatures. *Review of Educational Research* 61, 3 (1991), 287–292.
- [147] Sara Pérez-Soler, Esther Guerra, and Juan de Lara. 2018. Collaborative modeling and group decision making using chatbots in social networks. *IEEE Software* 35, 6 (2018), 48–54.
- [148] Kai Petersen and Claes Wohlin. 2009. Context in industrial software engineering research. In *2009 3rd international symposium on empirical software engineering and measurement*. IEEE, 401–404.
- [149] Burr Settles. 2009. *Active learning literature survey*. University of Wisconsin-Madison Department of Computer Sciences.
- [150] Amir Shevat. 2017. *Designing bots: Creating conversational experiences*. " O'Reilly Media, Inc".
- [151] Margaret-Anne Storey and Alexey Zagalsky. 2016. Disrupting developer productivity one bot at a time. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*. 928–931.
- [152] Sinarwati Mohamad Suhaili, Naomie Salim, and Mohamad Nazim Jambli. 2021. Service chatbots: A systematic review. *Expert Systems with Applications* 184 (2021), 115461.
- [153] Damian A Tamburri. 2019. Software architecture social debt: Managing the incommunicability factor. *IEEE Transactions on Computational Social Systems* 6, 1 (2019), 20–37.
- [154] Richard H Thaler and Cass R Sunstein. 2009. *Nudge: Improving decisions about health, wealth, and happiness*. Penguin.
- [155] Richard H Thaler, Cass R Sunstein, and John P Balz. 2013. *Choice architecture*. Vol. 2013. Princeton University Press Princeton, NJ.
- [156] Sahil Verma and Julia Rubin. 2018. Fairness definitions explained. In *2018 ieee/acm international workshop on software fairness (fairware)*. IEEE, 1–7.
- [157] Mairieli Wessel, Igor Wiese, Igor Steinmacher, and Marco Aurelio Gerosa. 2021. Don't Disturb Me: Challenges of Interacting with Software Bots on Open Source Software Projects. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW2 (2021), 1–21.
- [158] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Springer Science & Business Media.